

Algoritmos Recursivos en Teoría de Grafos

1. De qué trata este libro

1.1. Antecedentes

Un algoritmo es un método para resolver una clase de problemas en una computadora. La complejidad de un algoritmo es el costo, medido en tiempo de ejecución, almacenamiento, o cualquier otra unidad relevante, de utilizar el algoritmo para resolver uno de esos problemas.

Este libro trata sobre algoritmos y complejidad y, por lo tanto, sobre los métodos para resolver problemas en computadoras y los costos (usualmente el tiempo de ejecución) de emplear dichos métodos.

Computar requiere tiempo. Algunos problemas toman muchísimo tiempo; otros pueden realizarse rápidamente. Algunos problemas parecen tardar mucho y, de pronto, alguien descubre una manera más veloz de resolverlos (un «algoritmo más rápido»). El estudio de la cantidad de esfuerzo computacional necesario para llevar a cabo ciertos tipos de cálculos es el estudio de la complejidad computacional.

Naturalmente, esperaríamos que un problema de cómputo para el cual se requieran millones de bits de datos de entrada tarde probablemente más que otro problema que solo necesita unos pocos elementos de entrada. Así, la complejidad temporal de un cálculo se mide expresando el tiempo de ejecución del cálculo como una función de alguna medida de la cantidad de datos necesaria para describir el problema a la computadora.

Por ejemplo, considere la siguiente afirmación: «Acabo de comprar un programa para invertir matrices y puede invertir una matriz $n \times n$ en tan solo $1,2n^3$ minutos». Aquí observamos una descripción típica de la complejidad de un algoritmo determinado: el tiempo de ejecución del programa se da como una función del tamaño de la matriz de entrada.

Un programa más rápido para la misma tarea podría ejecutarse en $0,8n^3$ minutos para una matriz $n \times n$. Si alguien realizara un descubrimiento realmente importante (véase la Sección 2.4), entonces tal vez podríamos reducir el exponente en lugar de mantenerlo en 3.

meramente reduciendo la constante multiplicativa. Así, un programa que invertiese una matriz de $n \times n$ en tan solo $7n^{2,8}$ minutos supondría una mejora sorprendente del estado del arte.

A los efectos de este libro, una computación que está garantizada para tomar como máximo cn^3 tiempo para entrada de tamaño n se considerará una computación «fácil». Una que requiera como máximo n^{10} tiempo también es fácil. Si cierto cálculo sobre una matriz de $n \times n$ requiriese 2^n minutos, entonces sería un problema «difícil». Naturalmente, algunas de las computaciones que llamamos «fáciles» pueden tardar muchísimo en ejecutarse, pero aún así, desde nuestro punto de vista actual, la distinción importante será la garantía de tiempo polinómico o su ausencia.

La regla general es que si el tiempo de ejecución es como máximo una función polinómica de la cantidad de datos de entrada, entonces el cálculo es fácil; de lo contrario, es difícil.

Muchos problemas en informática son conocidos por ser fáciles. Para convencer a alguien de que un problema es fácil, basta con describir un método rápido para resolverlo. Convencer a alguien de que un problema es difícil es más complicado, pues habrá que demostrarle que es imposible encontrar una forma rápida de realizar el cálculo. No bastará con señalar un algoritmo particular y lamentar su lentitud. Al fin y al cabo, ese algoritmo puede ser lento, pero quizá exista uno más rápido.

La inversión de matrices es fácil. El método familiar de eliminación gaussiana puede invertir una matriz de $n \times n$ en tiempo como máximo cn^3 .

Para dar un ejemplo de un problema computacional difícil, debemos alejarnos un poco. Uno interesante se llama el «problema del enlosado». Supongamos¹ que disponemos de infinitas baldosas idénticas, cada una con forma de hexágono regular. Entonces podemos enlosar todo el plano con ellas, es decir, cubrir el plano sin dejar espacios vacíos. Esto también puede hacerse si las baldosas son rectángulos idénticos, pero no si son pentágonos regulares.

En la Figura 0.1 mostramos un enlosado del plano mediante rectángulos idénticos, y en la Figura 0.2 un enlosado mediante hexágonos regulares.

Eso plantea ciertas cuestiones teóricas y computacionales. Una pregunta computacional es la siguiente: supongamos que nos dan un cierto polígono, no necesariamente regular ni convexo, y que tenemos infinitas baldosas idénticas con esa forma. ¿Podremos o no conseguir enlosar todo el plano?

Esa elegante cuestión ha sido demostrada² como computacionalmente insoluble. En otras palabras, no solo desconocemos un modo rápido de resolverla, sino que sabemos que tal modo no puede existir.

¹Véase, por ejemplo, el artículo de Martin Gardner en *Scientific American*, enero de 1977, pp. 110–121.

²R. Berger, *The undecidability of the domino problem*, *Memoirs Amer. Math. Soc.* 66 (1966), Amer. Math. Soc., Providence, RI.

Cuando se presenta un problema de enlosado a una computadora, se ha demostrado que no existe ninguna forma de resolverlo, por lo que incluso buscar un algoritmo sería infructuoso. Eso no significa que la cuestión sea difícil para todo polígono. Los problemas difíciles pueden tener instancias fáciles. Lo que se ha demostrado es que no existe un método único que garantice que decidirá esta cuestión para cada polígono.

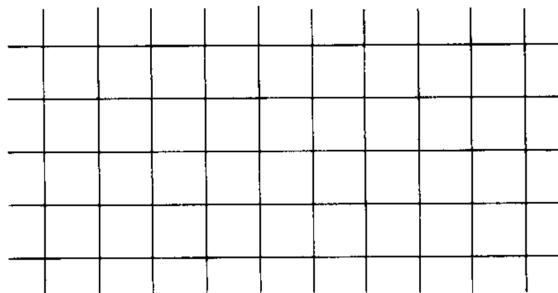


Figura 1: Enlosado con rectángulos.

El hecho de que un problema computacional sea difícil no significa que cada instancia del mismo deba ser difícil. El problema es difícil porque no podemos idear un algoritmo al que podamos otorgar una garantía de rendimiento rápido para todas las instancias.

Nótese que la cantidad de datos de entrada para la computadora en este ejemplo es bastante pequeña. Todo lo que necesitamos introducir es la forma del polígono básico. Sin embargo, no solo es imposible elaborar un algoritmo rápido para este problema, sino que se ha demostrado que es imposible idear cualquier algoritmo que esté garantizado a terminar con una respuesta Sí/No después de un número finito de pasos. ¡Eso sí que es difícil!

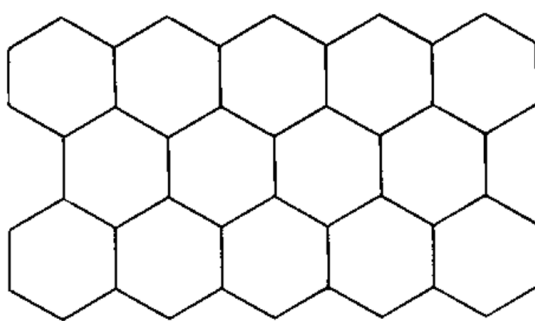


Figura 2: Enlosado con hexágonos.

2. Problemas difíciles frente a fáciles

Tomemos un momento más para decir, de otra manera, exactamente qué queremos decir con un cómputo «fácil» frente a uno «difícil».

Piensa en un algoritmo como una pequeña caja que puede resolver una cierta clase de problemas computacionales. En la caja entra la descripción de un problema particular de esa clase y entonces, tras cierta cantidad de tiempo o esfuerzo computacional, aparece la respuesta.

Ejemplo 0.1. Se garantiza que si el problema de entrada se describe con B bits de datos, entonces se obtendrá una respuesta después, como máximo, de $6B^3$ minutos.

Ejemplo 0.2. Se garantiza que todo problema cuya descripción requiera como máximo B bits de datos se resolverá en, como mucho, $0,7B^{15}$ segundos.

Una garantía de rendimiento, como las dos anteriores, a veces se denomina *estimación de complejidad del peor caso*, y es fácil ver por qué. Si tenemos un algoritmo que, por ejemplo, ordena cualquier secuencia dada de números en orden ascendente (ver Sección 2.2), puede ocurrir que algunas secuencias sean más fáciles de ordenar que otras.

Por ejemplo, la secuencia 1, 2, 7, 11, 10, 15, 20 ya está casi ordenada, así que nuestro algoritmo podría, si aprovecha ese orden parcial, ordenarla muy rápidamente. Otras secuencias podrían ser mucho más difíciles de manejar y, por tanto, requerir más tiempo.

Así que, para algunos problemas cuya cadena de bits de entrada tiene B bits, el algoritmo podría operar en tiempo $6B$, en otros necesitaría, digamos, $10B \log B$ unidades de tiempo, y para aún otras instancias de longitud B bits, podría necesitar $5B^2$ unidades de tiempo para completar la tarea.

Entonces, ¿qué diría la tarjeta de garantía? Tendría que considerar la posibilidad más desfavorable; de lo contrario, la garantía no sería válida. Aseguraría al usuario que, si la instancia del problema de entrada puede describirse con B bits, entonces aparecerá una respuesta tras, como máximo, $5B^2$ unidades de tiempo. Por lo tanto, una garantía de rendimiento equivale a una estimación del peor escenario posible: el cálculo más largo que podría ocurrir si se introducen B bits al programa.

Los límites de peor caso son el tipo más común, pero existen otras clases de límites para el tiempo de ejecución. Podríamos dar, en cambio, un límite de caso promedio (ver Sección 5.7). Eso no garantizaría un rendimiento peor que tal o cual; declararía que si se promedia el rendimiento sobre todas las cadenas de bits posibles de B bits, entonces el tiempo de cálculo promedio será tal o cual (en función de B).

Ahora hablemos de la diferencia entre problemas computacionales fáciles y difíciles y entre algoritmos rápidos y lentos.

Una garantía que no asegurase un rendimiento «rápido» contendría alguna función de B que creciese más rápido que cualquier polinomio. Por ejemplo, e^B o $2^{\sqrt{B}}$, etc. Es la garantía de tiempo polinómico frente a la que no necesariamente lo es lo que marca la diferencia entre las clases de problemas fáciles y difíciles, o entre algoritmos rápidos y lentos. Es muy deseable trabajar con algoritmos tales que podamos proporcionar una garantía de rendimiento para su tiempo de ejecución que sea, como mucho, una función polinómica del número de bits de entrada.

Un algoritmo es lento si, para cualquier polinomio P que consideremos, existen valores arbitrariamente grandes de B y cadenas de datos de entrada de B bits que hacen que el algoritmo realice más de $P(B)$ unidades de trabajo.

Un problema computacional es *tratable* si existe un algoritmo rápido que resuelva todas sus instancias.

Un problema computacional es *intratable* si puede demostrarse que no existe ningún algoritmo rápido para él.

Ejemplo 0.3. Aquí hay un problema computacional familiar y un método, o algoritmo, para resolverlo. Veamos si el método tiene una garantía de tiempo polinómico.

El problema es el siguiente. Sea n un número entero dado. Queremos averiguar si n es primo. El método que elegimos es el siguiente. Para cada entero

$$m = 2, 3, \dots, \lfloor \sqrt{n} \rfloor$$

preguntamos si m divide a n . Si todas las respuestas son «No», entonces declaramos que n es primo; en caso contrario, es compuesto.

Ahora analizaremos la complejidad computacional de este algoritmo. Eso significa que veremos cuántas operaciones implica la prueba. Para un entero dado n , el trabajo se puede medir en unidades de división de un número entero por otro. En esas unidades, obviamente realizaremos alrededor de $\sqrt{n}$ divisiones.

Parecería que este es un problema tratable, porque, al fin y al cabo, $\sqrt{n}$ crece de forma polinómica en n . Por ejemplo, hacemos menos de n divisiones, y eso sin duda es un polinomio en n . Según nuestra definición de algoritmos rápidos y lentos, la distinción se basó en el crecimiento polinómico frente al crecimiento más rápido que el polinómico del trabajo según el tamaño del problema, y, por tanto, este problema debería ser fácil. ¿Verdad? Pues no, en realidad no.

Conviene referirse a la distinción entre métodos rápidos y lentos para ver que debemos medir la cantidad de trabajo realizado en función del número de bits de entrada al problema. En este ejemplo, n no es el número de bits de la entrada. Por ejemplo, si $n = 59$, no necesitamos 59 bits para describir n , sino solo 6. En general, el número de dígitos binarios en la representación de un entero n es aproximadamente $\log_2 n$.

Así, en el problema de este ejemplo, el de probar la primalidad de un entero dado n , la longitud de la cadena de bits de entrada B es aproximadamente $\log_2 n$. Vista bajo esta perspectiva, el cálculo parece de pronto muy largo. Una cadena formada por tan solo $\log_2 n$ ceros y unos ha provocado que nuestro poderoso ordenador realice alrededor de $\sqrt{n}$ unidades de trabajo.

Si expresamos la cantidad de trabajo realizada como una función de B , encontramos que la complejidad de este cálculo es aproximadamente $2^{B/2}$, y eso crece mucho más rápido que cualquier función polinómica de B .

Por lo tanto, el método que acabamos de discutir para comprobar la primalidad de un entero dado es lento. La historia de este problema acaba de tomar un giro interesante. Tras muchos años en los que no se sabía si la primalidad podía probarse en tiempo polinómico, en 2002 se halló un nuevo algoritmo que lo hace exactamente. Véase el Capítulo 4 para una discusión más detallada de este problema.

En este libro abordaremos algunos problemas fáciles y otros que aparentan ser difíciles. Es el “aparentan” lo que hace las cosas muy interesantes. Se trata de problemas para los que nadie ha encontrado un algoritmo rápido, pero tampoco nadie ha demostrado la imposibilidad de que exista. Debe añadirse que toda el área se investiga con vigor debido al atractivo y a la importancia de las numerosas preguntas sin respuesta que aún persisten.

Así pues, aunque todavía no sepamos muchas cosas que nos gustaría conocer en este campo, ¡no es por falta de intentarlo!

2.1. 0.3 A Preview

El Capítulo 1 contiene parte de la base matemática que se necesitará para nuestro estudio de los algoritmos. No se pretende que la lectura de este libro, o su uso como texto en un curso, deba comenzar necesariamente con el Capítulo 1. Probablemente sea mejor sumergirse directamente en el Capítulo 2 y, cuando se requieran habilidades o conceptos concretos, leer las secciones pertinentes del Capítulo 1. De lo contrario, las definiciones e ideas de ese capítulo podrían parecer carentes de motivación cuando, de hecho, la motivación —en gran cantidad— reside en los capítulos posteriores del libro.

El Capítulo 2 trata sobre algoritmos recursivos y el análisis de sus complejidades.

El Capítulo 3 versa sobre un problema que parece difícil, pero resulta ser sencillo: el problema de flujo en redes. Gracias a investigaciones muy recientes, existen algoritmos rápidos para los problemas de flujo en redes, y estos tienen muchas aplicaciones importantes.

En el Capítulo 4 estudiamos algoritmos en una de las ramas más antiguas de las matemáticas, la teoría de números. Es notable que las conexiones entre esta

Las conexiones entre este tema antiguo y la investigación más moderna en métodos computacionales son muy fuertes.

En el Capítulo 5 veremos que existe una gran familia de problemas, que incluye varias cuestiones computacionales muy importantes, unidas por una notable unidad estructural. No sabemos si son difíciles o fáciles. Sí sabemos que aún no hemos encontrado una forma rápida de resolverlos, y la mayoría de la gente sospecha que son difíciles. También sabemos que, si alguno de estos problemas resulta ser difícil, entonces todos lo son, y si alguno resulta ser fácil, entonces todos lo son.

Esperamos que, después de haber descubierto algo sobre lo que las personas saben y lo que no saben, el lector haya disfrutado del recorrido por este tema y pueda estar interesado en ayudar a descubrir un poco más.

3. Preliminares Matemáticos

3.1. 1.1 Órdenes de magnitud

En esta sección vamos a analizar las tasas de crecimiento de distintas funciones y a presentar los cinco símbolos de la asintótica que se utilizan para describir esas tasas de crecimiento.

En el contexto de los algoritmos, el propósito de esta discusión es disponer de un buen lenguaje que nos permita comparar la rapidez con la que diferentes algoritmos realizan la misma tarea, la cantidad de memoria que utilizan o cualquier otra medida de complejidad que estemos empleando.

Supongamos que disponemos de un procedimiento para invertir matrices cuadradas no singulares. ¿Cómo podríamos medir su velocidad? Lo habitual sería decir algo como: «si la matriz es de tamaño $n \times n$, entonces el método se ejecuta en un tiempo $16,8 n^3$.» Con ello sabríamos que, si una matriz de 100×100 puede invertirse, con este método, en 1 minuto de tiempo de computador, entonces una matriz de 200×200 requeriría $2^3 = 8$ veces más tiempo, es decir, unos 8 minutos. La constante “16.8” no intervino en absoluto en este ejemplo; lo relevante fue únicamente que el trabajo crece como la tercera potencia del tamaño de la matriz.

Por tanto, necesitamos un lenguaje que nos permita afirmar que el tiempo de cómputo, como función de n , crece «del orden de n^3 », o «a lo sumo tan rápido como n^3 », o «al menos tan rápido como $n^5 \log n$ », etcétera.

Los símbolos que se emplean en este lenguaje de comparación de tasas de crecimiento son los siguientes cinco: « o » (se lee «es pequeña o de»), « O » (se lee «es gran O de»), « Θ » (se lee «es theta de»), « $\sim$ » (se lee «es asintóticamente igual a», o de manera menos formal, «twiddles») y « Ω » (se lee «es omega de»).

Ahora expliquemos qué significa cada uno de ellos. Sean $f(x)$ y $g(x)$ dos funciones de x . Cada uno de los cinco símbolos anteriores permite comparar la rapidez de crecimiento de f y g . Si decimos que

$$f(x) = o(g(x)),$$

entonces, de forma informal, estamos diciendo que f crece más lentamente que g cuando x es muy grande. De manera formal, la definición se enuncia así:

Definición 1.1. Decimos que $f(x) = o(g(x))$ ($x \rightarrow \infty$) si

$$\lim_{x \rightarrow \infty} \frac{f(x)}{g(x)} = 0.$$

Aquí van algunos ejemplos:

- (a) $x^2 = o(x^5)$
- (b) $\sin x = o(x)$
- (c) $14,709\sqrt{x} = o(x/2 + 7 \cos x)$
- (d) $\frac{1}{x} = o(1)$ (?)
- (e) $23 \log x = o(x^{0,02})$

A partir de estos pocos ejemplos se puede ver que, a veces, demostrar que existe una relación de tipo ‘ o ’ puede ser sencillo y, en otras ocasiones, bastante difícil. El ejemplo (e), por ejemplo, requiere el uso de la regla de *L’Hôpital*.

Si disponemos de dos programas de ordenador y uno de ellos invierte matrices $n \times n$ en tiempo $635n^3$, mientras que el otro lo hace en tiempo $o(n^{2,8})$, sabemos que, para valores suficientemente grandes de n , la garantía de rendimiento del segundo programa será superior a la del primero. Por supuesto, el primer programa podría ejecutarse más rápido en matrices pequeñas, digamos de hasta tamaño $10\,000 \times 10\,000$. Si cierto programa se ejecuta en tiempo $n^{2,03}$ y alguien produce otro programa para el mismo problema que corre en tiempo $o(n^2 \log n)$, entonces ese segundo programa supondría una mejora, al menos en el sentido teórico. La razón de la calificación “teórico”, una vez más, es que el segundo programa sería superior solo si n fuera suficientemente grande.

El segundo símbolo del vocabulario de la asintótica es la ‘ O ’. Cuando decimos que $f(x) = O(g(x))$ queremos decir, de forma informal, que f ciertamente no crece más rápido que g ; podría crecer a la misma velocidad o más despacio: ambas posibilidades las permite la ‘ O ’. De manera formal, tenemos la siguiente definición:

Definición 1.2. Decimos que $f(x) = O(g(x))$ ($x \rightarrow \infty$) si existen C, x_0 tales que

$$|f(x)| < Cg(x) \quad \text{para todo } x > x_0.$$

El calificativo “ $x \rightarrow \infty$ ” se omitirá normalmente, pues se entenderá que la mayoría de las veces estaremos interesados en valores grandes de las variables involucradas.

Por ejemplo, ciertamente $\sin x = O(x)$, pero aún se puede afirmar más, a saber, que $\sin x = O(1)$. Asimismo,

$$x^3 + 5x^2 + 77 \cos x = O(x^5).$$

y $1/(1+x^2) = O(1)$. Ahora podemos ver cómo la ‘ o ’ aporta información más precisa que la ‘ O ’, pues podemos afinar el último ejemplo diciendo que $1/(1+x^2) = o(1)$. Esto es más preciso porque no solo nos indica que la función está acotada cuando x es grande, sino que también aprendemos que la función en realidad se aproxima a 0 cuando $x \rightarrow \infty$.

Esto es típico de la relación entre O y o . Con frecuencia, un resultado en ‘ O ’ es suficiente para una aplicación. Sin embargo, es posible que no sea así y que necesitemos la estimación más precisa en ‘ o ’.

El tercer símbolo del lenguaje de la asintótica es el ‘ Θ ’.

Definición 1.3. Decimos que $f(x) = \Theta(g(x))$ si existen constantes $c_1 > 0$, $c_2 > 0$, x_0 tales que para todo $x > x_0$ se cumple $c_1 g(x) < f(x) < c_2 g(x)$.

Entonces podemos decir que f y g tienen la misma tasa de crecimiento; sólo las constantes multiplicativas son inciertas. Algunos ejemplos del uso de ‘ Θ ’ son:

$$\begin{aligned}(x+1)^2 &= \Theta(3x^2), \\ \frac{x^2+5x+7}{5x^3+7x+2} &= \Theta\left(\frac{1}{x}\right), \\ \sqrt{3+\sqrt{2}x} &= \Theta(x^{1/4}), \\ \left(1+\frac{3}{x}\right)^x &= \Theta(1).\end{aligned}$$

El ‘ Θ ’ es mucho más preciso que el ‘ O ’ o el ‘ o ’. Si sabemos que $f(x) = \Theta(x^2)$, entonces conocemos que $f(x)/x^2$ permanece entre dos constantes no nulas para valores de x suficientemente grandes. La tasa de crecimiento de f queda establecida: crece cuadráticamente con x .

El símbolo más preciso de la asintótica es el ‘ $\sim$ ’. Nos dice que no sólo f y g crecen a la misma tasa, sino que en realidad f/g se aproxima a 1 cuando $x \rightarrow \infty$.

Definición 1.4. Decimos que $f(x) \sim g(x)$ si

$$\lim_{x \rightarrow \infty} \frac{f(x)}{g(x)} = 1.$$

A continuación se muestran algunos ejemplos:

$$\begin{aligned}x^2 + x &\sim x^2, \\ (3x+1)^4 &\sim 81x^4, \\ \sin\left(\frac{1}{x}\right) &\sim \frac{1}{x}, \\ \frac{2x^3+5x+7}{x^2+4} &\sim 2x, \\ 2x + 7\log x + \cos x &\sim 2x.\end{aligned}$$

Observemos la importancia de precisar las constantes multiplicativas cuando se usa el símbolo ‘ $\sim$ ’. Si bien es cierto que $2x^2 = \Theta(x^2)$, no es cierto que $2x^2 \sim x^2$. Dicho sea de paso, también es cierto que $2x^2 = \Theta(17x^2)$, pero hacer tal afirmación es de mal estilo, pues el ‘17’ no aporta información adicional.

El último símbolo del lenguaje asintótico que necesitaremos es el ‘ Ω ’. En pocas palabras, ‘ Ω ’ es la negación de ‘ o ’. Es decir, $f(x) = \Omega(g(x))$ significa que no es cierto que $f(x) = o(g(x))$. En el estudio de algoritmos para computadoras, se emplea ‘ Ω ’ cuando queremos expresar que cierto cálculo requiere al menos una cantidad dada de tiempo.

Por ejemplo, podemos multiplicar dos matrices $n \times n$ en tiempo $O(n^3)$. Más adelante veremos cómo hacerlo aún más rápido, en $O(n^{2.81})$. Existen métodos incluso más veloces, pero algo es seguro: nadie podrá escribir un programa que multiplique pares de matrices $n \times n$ empleando menos de n^2 pasos computacionales, porque cualquier programa debe leer los datos de entrada y las matrices contienen $2n^2$ entradas. Por lo tanto, un tiempo de cómputo de cn^2 es, sin duda, una cota inferior para la velocidad de cualquier algoritmo general de multiplicación de matrices. Podemos decir, entonces, que el problema de multiplicar dos matrices $n \times n$ requiere $\Omega(n^2)$ tiempo.

La definición exacta de ‘ Ω ’ dada arriba es bastante delicada: la presentamos como la negación de algo. ¿Podemos reformularla de manera positiva? Sí, con algo de trabajo (véanse los Ejercicios 6 y 7). Puesto que $f = o(g)$ significa que $f/g \rightarrow 0$, afirmar $f = \Omega(g)$ implica que f/g *no* tiende a cero. Si suponemos que g toma solamente valores positivos (lo habitual), decir que f/g no tiende a 0 equivale a afirmar que existe $\varepsilon > 0$ y una sucesión infinita de valores de x , con $x \rightarrow \infty$, para los cuales $|f|/g > \varepsilon$. Por tanto, no necesitamos mostrar que $|f|/g > \varepsilon$ para todo x grande, sino únicamente para infinitos valores grandes de x .

Definición 1.5. Decimos que $f(x) = \Omega(g(x))$ si existen $\varepsilon > 0$ y una sucesión $x_1, x_2, x_3, \dots$, que tiende a infinito, tal que para todo j se cumple

$$|f(x_j)| > \varepsilon g(x_j).$$

Ahora presentaremos una jerarquía de funciones según sus tasas de crecimiento cuando x es grande. Entre las funciones de uso común que tienden a infinito al crecer x , quizá las más lentas sean $\log \log x$ o incluso $(\log \log x)^{1.03}$ y expresiones similares. Es cierto que $\log \log x \rightarrow \infty$ cuando $x \rightarrow \infty$, pero lo hace muy despacio: cuando $x = 1\,000\,000$, por ejemplo, $\log \log x = 2.6$.

Un poco más rápida que estas “babosas” es $\log x$ en sí. Al fin y al cabo, $\log 1\,000\,000 = 13.8$. De modo que, si tuviéramos un algoritmo de computadora que

Se podría realizar n operaciones en tiempo $\log n$ y alguien encontrara otro método que hiciera el mismo trabajo en tiempo $O(\log \log n)$; entonces, el segundo método, *ceteris paribus*, sería ciertamente una mejora, pero n tendría que ser extremadamente grande para que se notara dicha ventaja.

En la escala de rapidez de crecimiento, a continuación de los logaritmos, encontramos las potencias de x . Por ejemplo, consideremos $x^{0,01}$. Esta función crece más rápido que $\log x$, aunque no lo parecería si sustituyéramos algunos valores de x y comparáramos los resultados (véase el Ejercicio 1 al final de esta sección).

¿Cómo demostrar que $x^{0,01}$ crece más rápido que $\log x$? Mediante la regla de L'Hôpital.

Ejemplo 1.1. Consideremos el límite de $x^{0,01}/\log x$ cuando $x \rightarrow \infty$. A medida que $x \rightarrow \infty$, el cociente adopta la forma indeterminada ∞/∞ , de modo que podemos aplicar la regla de L'Hôpital: derivamos el numerador, derivamos el denominador y volvemos a tomar el límite $x \rightarrow \infty$. Así, en lugar del cociente original obtenemos

$$\frac{0,01 x^{-0,99}}{1/x} = 0,01 x^{0,01},$$

el cual, evidentemente, crece sin cota superior cuando $x \rightarrow \infty$. Por lo tanto, el cociente original $x^{0,01}/\log x$ también crece sin límite. Hemos demostrado, con precisión, que $\log x = o(x^{0,01})$; en ese sentido, podemos afirmar que $x^{0,01}$ crece más rápido que $\log x$.

Para continuar ascendiendo en la escala de tasas de crecimiento, nos encontramos con $x^{0,2}$, x , x^{15} , $x^{15} \log^2 x$, etcétera. Luego aparecen funciones que crecen más rápido que cualquier potencia fija de x , del mismo modo que $\log x$ crece más lento que toda potencia fija de x .

Consideremos $e^{\log^2 x}$. Como esto es lo mismo que $x^{\log x}$, crecerá más rápido que x^{1000} ; de hecho, será mayor que x^{1000} tan pronto como $\log x > 1000$, es decir, tan pronto como $x > e^{1000}$ (¡no contengas la respiración!).

Así pues, $e^{\log^2 x}$ es un ejemplo de función que crece más rápido que toda potencia fija de x . Otro ejemplo es $e^{\sqrt{x}}$ (¿por qué?).

Definición 1.6. Decimos que una función presenta *crecimiento exponencial moderado* si crece más rápido que x^a , para toda constante a , pero más lento que c^x , para toda constante $c > 1$. Más precisamente, $f(x)$ es de crecimiento exponencial moderado si para todo $a > 0$ se cumple $f(x) = \Omega(x^a)$ y para todo $\varepsilon > 0$ se cumple $f(x) = o((1 + \varepsilon)^x)$.

Más allá del rango de crecimiento exponencial moderado están las funciones que crecen exponencialmente rápido. Ejemplos típicos de tales funciones son $(1,03)^x$, 2^x , x^{97x} , y así sucesivamente. Formalmente, tenemos:

Definición 1.7. Una función f es de *crecimiento exponencial* si existe $c > 1$ tal que $f(x) = \Omega(c^x)$ y existe d tal que $f(x) = O(d^x)$.

Si “adornamos” una función de crecimiento exponencial con funciones menores, no cambiaremos el hecho de que sigue siendo de crecimiento exponencial. Así,

$$e^{\sqrt{x}+2x/(x^{49}+37)}$$

continúa siendo de crecimiento exponencial, porque e^{2x} ya lo es por sí misma y resiste los intentos de las funciones menores por alterar su comportamiento.

Más allá de las funciones de crecimiento exponencial existen funciones que crecen tan rápido como se desee. Por ejemplo, $n!$, que crece más rápido que c^n para toda constante fija c , y 2^{n^2} , que crece mucho más rápido que $n!$. Los rangos de crecimiento que más preocupan a los científicos de la computación están “entre” las funciones logarítmicas, muy lentas, y las de crecimiento exponencial. La razón es simple: si un algoritmo requiere más de una cantidad exponencial de tiempo para realizar su tarea, probablemente no se usará, o se usará solo en circunstancias muy inusuales. En este libro todos los algoritmos que veremos caen dentro de ese rango intermedio.

Ya hemos discutido los distintos símbolos asintóticos que se emplean para comparar tasas de crecimiento de pares de funciones y la jerarquía de rapidez de crecimiento, de modo que contamos con un pequeño catálogo de funciones que crecen lentamente, medianamente rápido, rápido y muy rápido. A continuación, estudiaremos el crecimiento de sumas que involucran funciones elementales, con el fin de descubrir a qué ritmo crecen dichas sumas.

Pensemos en la siguiente expresión:

$$f(n) = \sum_{j=0}^n j^2 = 1^2 + 2^2 + 3^2 + \cdots + n^2. \quad (1.1)$$

Es decir, $f(n)$ es la suma de los cuadrados de los primeros n enteros positivos.

¿Qué tan rápido crece $f(n)$ cuando n es grande?

Observemos de inmediato que, entre los n términos de la suma que define $f(n)$, el mayor es el último, a saber, n^2 . Como hay n términos y el mayor vale apenas n^2 , es evidente que $f(n) = O(n^3)$, e incluso que $f(n) \leq n^3$ para todo $n \geq 1$.

Supongamos que deseamos información más precisa sobre el crecimiento de $f(n)$, por ejemplo una afirmación del estilo $f(n) \sim ?$. ¿Cómo podríamos lograr una estimación mejor?

La mejor manera de empezar es visualizar la suma en (1.1), como se muestra en la Figura 1.1. En la figura se ve la curva $y = x^2$ en el plano x - y . Además, se dibuja un rectángulo sobre cada intervalo de longitud unitaria desde $x = 1$ hasta $x = n$. Todos los rectángulos quedan bajo la curva.

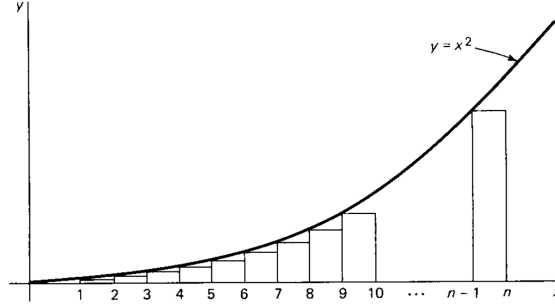


Figura 3: Cómo sobreestimar una suma.

En consecuencia, el área total de todos los rectángulos es menor que el área bajo la curva, lo cual se expresa como:

$$\sum_{j=1}^{n-1} j^2 \leq \int_1^n x^2 dx = \frac{n^3 - 1}{3}. \quad (1.2)$$

Si comparamos (??) con (??), observamos que hemos demostrado que

$$f(n) \leq \frac{(n+1)^3 - 1}{3}.$$

Ahora obtendremos una cota inferior para $f(n)$ de la misma manera. Esta vez empleamos la configuración de la Figura 1.2, donde nuevamente aparece la curva $y = x^2$, pero los rectángulos se han dibujado *encima* de la curva. A partir de la imagen vemos de inmediato que

$$1^2 + 2^2 + \cdots + n^2 \geq \int_0^n x^2 dx = \frac{n^3}{3}. \quad (1.3)$$

Hemos acotado nuestra función $f(n)$ por ambos lados, y de forma bastante estrecha. Sabemos que

$$\forall n \geq 1 : \quad \frac{n^3}{3} \leq f(n) \leq \frac{(n+1)^3 - 1}{3}.$$

De aquí obtenemos inmediatamente que $f(n) \sim n^3/3$, lo que nos brinda una idea bastante precisa de la tasa de crecimiento de $f(n)$ cuando n es grande. El lector habrá notado también que el símbolo ' $\sim$ ' ofrece una estimación de crecimiento mucho más satisfactoria que la de ' O '.

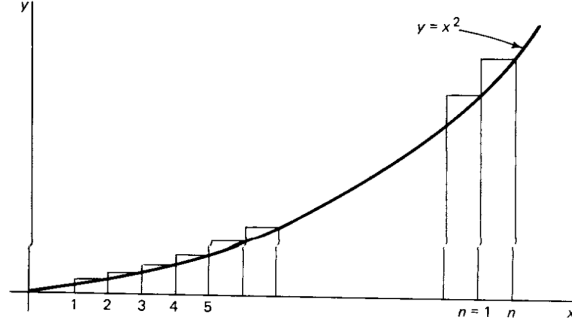


Figura 4: Cómo subestimar una suma.

Formulemos un principio general, para estimar el tamaño de una suma, que produzca estimaciones como las anteriores sin exigirnos cada vez visualizar figuras como las Figuras 1.1 y 1.2. La idea general es que, cuando se enfrenta la tarea de estimar las tasas de crecimiento de las sumas, se debe intentar comparar las sumas con integrales, porque normalmente son más fáciles de manejar.

Sea una función $g(n)$ definida para valores enteros no negativos de n , y supongamos que $g(n)$ es no decreciente. Queremos estimar el crecimiento de la suma:

$$G(n) = \sum_{j=1}^n g(j) \quad (n = 1, 2, \dots). \quad (1.4)$$

Considérese un diagrama que luce exactamente como la Figura 1.1 salvo que la curva mostrada es ahora la curva $y = g(x)$. La suma de las áreas de los rectángulos es exactamente $G(n-1)$, mientras que el área bajo la curva entre 1 y n es $\int_1^n g(t) dt$. Puesto que los rectángulos quedan totalmente debajo de la curva, sus áreas combinadas no pueden exceder el área bajo la curva, y tenemos la desigualdad:

$$G(n-1) \leq \int_1^n g(t) dt \quad (n \geq 1). \quad (1.5)$$

Por otro lado, si consideramos la Figura 1.2, donde la gráfica es de nuevo la gráfica de $y = g(x)$, el hecho de que las áreas combinadas de los rectángulos no sean ahora menores que el área bajo la curva produce la desigualdad:

$$G(n) \geq \int_0^n g(t) dt \quad (n \geq 1). \quad (1.6)$$

Si combinamos (1.5) y (1.6) vemos que hemos completado la demostración de:

Teorema 1. *Sea $g(x)$ no decreciente para x no negativos. Entonces*

$$\int_0^n g(t) dt \leq \sum_{j=1}^n g(j) \leq \int_1^{n+1} g(t) dt. \quad (1.7)$$

El teorema anterior es capaz de producir estimaciones bastante satisfactorias con muy poco esfuerzo, como muestra el siguiente ejemplo.

Sea $g(n) = \log n$ y sustitúyase en (1.7). Tras efectuar las integrales, obtenemos:

$$n \log n - n \leq \sum_{j=1}^n \log j \leq (n+1) \log(n+1) - n. \quad (1.8)$$

Reconocemos que el término central anterior es $\log n!$, y por lo tanto, al exponentiar (1.8) tenemos

$$\left(\frac{n}{e}\right)^n \leq n! \leq \frac{(n+1)^{n+1}}{e^n}. \quad (1.9)$$

Esta es una estimación bastante buena del crecimiento de $n!$, pues el término derecho es solo aproximadamente ne veces mayor que el izquierdo (¿por qué?), cuando n es grande.

Mediante el uso de herramientas ligeramente más precisas puede demostrarse una mejor estimación del tamaño de $n!$ llamada *fórmula de Stirling*, que establece que:

$$x! \sim \left(\frac{x}{e}\right)^x \sqrt{2\pi x}. \quad (1.10)$$

3.1.1. Ejercicios 1.1.1

1. Calcule los valores de $x^{0,01}$ y de $\log x$ para $x = 10, 1000, 1\,000\,000$. Encuentre un valor único de $x > 10$ para el cual $x^{0,01} > \log x$, y demuestre que su respuesta es correcta.
2. Algunas de las siguientes afirmaciones son verdaderas y otras falsas. ¿Cuáles son cuáles?

(a) $(x^2 + 3x + 1)^3 \sim x^6$

(b) $\frac{(\sqrt{x} + 1)^3}{x^2 + 1} = o(1)$

- (a) $e^{1/x} = \Theta(1)$
- (b) $\frac{1}{x} \sim 0$
- (c) $x^3(\log \log x)^2 = o(x^3 \log x)$
- (d) $\sqrt{\log x} + 1 = \Omega(\log \log x)$
- (e) $\sin x = \Omega(1)$
- (f) $\frac{\cos x}{x} = O(1)$
- (g) $\int_4^x \frac{dt}{t} \sim \log x$
- (h) $\int_0^x e^{-t^2} dt = O(1)$
- (i) $\sum_{j \leq x} \frac{1}{j^2} = o(1)$
- (j) $\sum_{j \leq x} 1 \sim x$

3. Cada una de las tres sumas siguientes define una función de x . Bajo cada suma aparece una lista de cinco afirmaciones sobre la tasa de crecimiento, cuando $x \rightarrow \infty$, de la función que la suma define. En cada caso, indique cuál (o cuáles) de las cinco opciones es verdadera (*puede haber más de una verdadera*).

- $h_1(x) = \sum_{j \leq x} \left(\frac{1}{j} + \frac{3}{j^2} + \frac{4}{j^3} \right)$

- (a) $\sim \log x$
- (b) $= O(x)$
- (c) $\sim 2 \log x$
- (d) $= \Theta(\log x)$
- (e) $= \Omega(1)$

- $h_2(x) = \sum_{j \leq \sqrt{x}} (\log j + j)$

- (a) $\sim x/2$
- (b) $= O(\sqrt{x})$
- (c) $= \Theta(\sqrt{x} \log x)$
- (d) $= \Omega(\sqrt{x})$
- (e) $= o(\sqrt{x})$

- $h_3(x) = \sum_{j \leq \sqrt{x}} \frac{1}{\lceil j \rceil}$

$$(a) = O(\sqrt{x})$$

$$(b) = \Omega(x^{1/4})$$

$$(c) = o(x^{1/4})$$

$$(d) \sim 2x^{1/4}$$

$$(e) = \Theta(x^{1/4})$$

4. De los cinco símbolos de asintótica O , o , $\sim$, Θ , Ω , ¿cuáles son transitivos? (por ejemplo, si $f = O(g)$ y $g = O(h)$, entonces $f = O(h)$?)
5. El objetivo de este ejercicio es mostrar que, si f crece más lentamente que g , siempre podemos encontrar una tercera función h cuya tasa de crecimiento se sitúe entre la de f y la de g . Demuestre lo siguiente: si $f = o(g)$, entonces existe una función h tal que $f = o(h)$ y $h = o(g)$. Dé una construcción explícita de h en términos de f y de g .
6. *Calentamiento para el ejercicio 7.* A continuación aparecen varias proposiciones matemáticas. En cada caso, escriba una proposición que sea la negación de la dada.

Además, en la negación, no utilices la palabra «no» ni ningún símbolo de negación. En cada caso la pregunta es: “Si esto no es verdadero, ¿entonces qué es verdadero?”

- (a) $\exists x > 0 \ni f(x) = 0$
- (b) $\forall x > 0, f(x) > 0$
- (c) $\forall x > 0, \exists \varepsilon > 0 \ni f(x) < \varepsilon$
- (d) $\exists x = 0 \ni \forall y < 0, f(y) < f(x)$
- (e) $\forall x \exists y \ni \forall z : g(x) < f(y)f(z)$
- (f) $\forall \varepsilon > 0 \exists x \ni \forall y > x : f(y) < \varepsilon$

¿Puedes formular un método general para negar tales proposiciones? Dada una proposición que contenga « $\forall$ », « $\exists$ », « $\ni$ », ¿qué regla aplicarías para negarla y dejar el resultado en forma positiva (sin símbolos de negación ni «no»)?

1. En este ejercicio elaboraremos la definición de ‘ Ω ’.

- (a) Escribe la definición precisa de la afirmación $\lim_{x \rightarrow \infty} h(x) = 0$ (usa ε ’s).
- (b) Escribe la negación de tu respuesta al apartado (a) como una afirmación positiva.
- (c) Utiliza tu respuesta al apartado (b) para dar una definición positiva de la afirmación $f(x) = o(g(x))$ y, con ello, justifica la definición del símbolo ‘ Ω ’ dada en el texto.

- 2. Ordena las siguientes funciones de menor a mayor tasa de crecimiento para n grande, de modo que cada una sea ‘little-oh’ de su sucesora: $2^{\sqrt{n}}$, $e^{\log n^3}$, $n^{3,01}$, 2^{n^2} , $n^{1,6}$, $\log n^3 + 1$, $\sqrt{n}!$, $n^3 \log n$, $n^{\log n}$, $(\log \log n)^3$, $n^{0,5} 2^n$, $(n+4)^{12}$.
- 3. Encuentra una función $f(x)$ tal que $f(x) = \mathcal{O}(x^{1+\varepsilon})$ sea verdadera para todo $\varepsilon > 0$, pero para la cual no sea cierto que $f(x) = \mathcal{O}(x)$.
- 4. Demuestra que la afirmación $f(n) = \mathcal{O}((2+\varepsilon)^n)$ para todo $\varepsilon > 0$ es equivalente a la afirmación $f(n) = o((2+\varepsilon)^n)$ para todo $\varepsilon > 0$.

3.2. Sistemas numéricos posicionales

En esta sección se ofrece una breve revisión de la representación de números en distintas bases. El sistema decimal habitual representa los números utilizando los

dígitos $0, 1, \dots, 9$. Con el propósito de representar números enteros, podemos imaginar que las potencias de 10 se muestran ante nosotros de la siguiente forma:

$$\dots, 100000, 10000, 1000, 100, 10, 1.$$

A continuación, para representar un número entero, podemos especificar cuántas copias de cada potencia de 10 deseamos. Si escribimos 237, por ejemplo, significa que queremos 2 centenas, 3 decenas y 7 unidades.

En general, si escribimos la cadena de dígitos que representa un número en el sistema decimal, como $d_m d_{m-1} \dots d_1 d_0$, entonces el número representado por dicha cadena es

$$n = \sum_{i=0}^m d_i 10^i.$$

Probemos ahora con el sistema binario. En lugar de usar 10, utilizaremos 2. Por lo tanto, imaginamos que las potencias de 2 se muestran ante nosotros como:

$$\dots, 512, 256, 128, 64, 32, 16, 8, 4, 2, 1.$$

Para representar un número, especificaremos cuántas copias de cada potencia de 2 queremos. Por ejemplo, si escribimos 1101, entonces necesitamos un 8, un 4 y un 1; por lo tanto, este debe ser el número decimal 13. Escribiremos

$$(13)_{10} = (1101)_2$$

para indicar que el número 13 en base 10 es el mismo que 1101 en base 2.

En el sistema binario (base 2) los únicos dígitos necesarios son 0 y 1. Esto significa que, si usamos solo 0 y 1, podremos representar cada número n de manera única. La representación única de cada número es, después de todo, lo que debemos esperar y exigir de cualquier sistema propuesto.

Profundicemos en este último punto. Si se nos permitiera usar más dígitos que solo 0 y 1, entonces podríamos representar el número $(13)_{10}$ en binario de muchas maneras. Por ejemplo, podríamos cometer el error de permitir los dígitos 0, 1, 2, 3. Entonces 13 sería representable por $3 \cdot 2^2 + 1 \cdot 2^0$ o por $2 \cdot 2^2 + 2 \cdot 2^1 + 1 \cdot 2^0$, etc.

Si permitiéramos demasiados dígitos distintos, los números serían representables de más de una forma mediante una cadena de dígitos.

Si, en cambio, permitiéramos muy pocos dígitos, descubriríamos que algunos números no tendrían representación alguna. Por ejemplo, si usáramos el sistema decimal con solo los dígitos $0, 1, \dots, 8$, entonces infinitos números no podrían representarse, así que será mejor mantener el 9.

La proposición general es la siguiente:

Teorema 1.9. Sea $b > 1$ un entero positivo (la *base*). Entonces todo entero positivo n puede escribirse de una y sólo una forma como

$$n = d_0 + d_1b + d_2b^2 + d_3b^3 + \dots$$

si los dígitos $d_0, d_1, \dots$ satisfacen $0 \leq d_i \leq b - 1$ para todo i .

Observación. El teorema afirma, por ejemplo, que en base 10 necesitamos los dígitos $0, 1, \dots, 9$; en base 2 necesitamos sólo 0 y 1; en base 16 necesitamos dieciséis dígitos, etc.

Demostración. Fijada la base b , la prueba es por inducción sobre n , el número que se representa. Es claro que el número 1 puede representarse de una y sólo una forma con los dígitos disponibles (¿por qué?). Supongamos inductivamente que cada entero $1, 2, \dots, n-1$ es representable de manera única. Consideremos ahora el entero n . Definamos

$$d = n \text{ mód } b.$$

Entonces d es uno de los b dígitos permitidos. Por hipótesis de inducción, el número

$$n_0 = \frac{n - d}{b}$$

es representable de manera única; esto es,

$$\frac{n - d}{b} = d_0 + d_1b + d_2b^2 + \dots$$

Por lo tanto,

$$n = d + \frac{n - d}{b}b = d + d_0b + d_1b^2 + d_2b^3 + \dots$$

es una representación de n que utiliza sólo los dígitos permitidos.

Supongamos, finalmente, que n posee otra representación de esta forma. Entonces

$$n = a_0 + a_1b + a_2b^2 + \dots = c_0 + c_1b + c_2b^2 + \dots$$

Como a_0 y c_0 son ambos iguales a $n \text{ mód } b$, concluimos que $a_0 = c_0$. En consecuencia,

$$n_0 = \frac{n - a_0}{b}$$

tendría dos representaciones diferentes, contradiciendo la hipótesis inductiva, pues hemos supuesto la veracidad del resultado para todo $n_0 < n$. $\square$

Los valores típicos de b más utilizados, además del 10, son 2 (*sistema binario*), 8 (*sistema octal*) y 16 (*sistema hexadecimal*).

El sistema binario es extremadamente sencillo porque utiliza solo dos dígitos. Esto es muy conveniente si eres una computadora o un diseñador de computadoras, pues los dígitos pueden determinarse según que un componente esté ‘encendido’ (dígito 1) o ‘apagado’ (dígito 0). Los dígitos binarios de un número se llaman sus *bits* o su *cadena de bits*.

El sistema octal es popular porque ofrece una buena manera de recordar y manejar las largas cadenas de bits que produce el sistema binario. De acuerdo con el teorema, en el sistema octal los dígitos necesarios son $0, 1, \dots, 7$. Por ejemplo:

$$(735)_8 = (477)_{10}.$$

La característica atractiva del sistema octal es la facilidad con la que podemos convertir entre octal y binario. Si se nos da la cadena de bits de un entero n , para convertirla a octal basta con agrupar los bits en grupos de tres, empezando por el bit menos significativo, y luego convertir cada grupo de tres bits, de forma independiente, en un único dígito octal. A la inversa, si se nos da la forma octal de n , la forma binaria se obtiene convirtiendo cada dígito octal, de manera independiente, en los tres bits que lo representan en el sistema binario. Por ejemplo, dado $(1101100101)_2$. Para convertir este número binario a octal, agrupamos los bits de tres en tres,

$$(1)(101)(100)(101),$$

comenzando desde la derecha, y luego convertimos cada triple en un único dígito octal, obteniendo así:

$$(1101100101)_2 = (1545)_8.$$

Si eres un programador en activo, resulta muy práctico usar las cadenas octales más cortas para recordar o anotar las cadenas binarias más largas, debido al ahorro de espacio unido a la facilidad de conversión en ambos sentidos.

El sistema hexadecimal (base 16) es como el octal, pero aún más. La conversión de ida y vuelta a binario ahora utiliza grupos de *cuatro* bits, en lugar de tres. En hexadecimal necesitaremos, según el teorema anterior, 16 dígitos. Disponemos de nombres prácticos para los primeros 10, pero ¿cómo denominaremos a los ‘dígitos 10 a 15’? Los nombres que se usan convencionalmente para ellos son ‘A’, ‘B’, $\dots$, ‘F’. Por ejemplo:

$$(A52C)_{16} = 10(4096) + 5(256) + 2(16) + 12 = (42284)_{10}.$$

Por otro lado, si convertimos cada dígito hexadecimal, de manera independiente, a base 2, encontramos que

$$(A52C)_{16} = (1010)_2(0101)_2(0010)_2(1100)_2 = (1010010100101100)_2.$$

Finalmente, si **agrupamos los bits en grupos de tres**, podemos convertir a octal del modo siguiente:

$$(1010010100101100)_2 = (1)(010)(010)(100)(101)(100) = (122454)_8.$$

3.2.1. Ejercicios 1.2.1

1. Demuestre que la conversión de octal a binario se realiza correctamente convirtiendo cada dígito octal en un trío binario y concatenando los tríos resultantes. Generalice este teorema a otros pares de bases.
2. Efectúe las conversiones indicadas.
 - (a) $(737)_{10} = ?_3$
 - (b) $(101100)_2 = ?_{16}$
 - (c) $(3377)_8 = ?_{16}$
 - (d) $(ABCD)_{16} = ?_{10}$
 - (e) $(BEEF)_{16} = ?_8$
3. Escriba un procedimiento `convert(n, b:integer, digitstr:string)` que devuelva la cadena de dígitos que representa n en la base b .

3.3. Manipulaciones con series

En esta sección estudiaremos operaciones con series de potencias, incluyendo su multiplicación y la obtención de sus sumas en forma sencilla. Comenzamos con un pequeño catálogo de series de potencias que conviene conocer. Primero tenemos la *serie geométrica finita*:

$$\frac{1 - x^n}{1 - x} = 1 + x + x^2 + \cdots + x^{n-1}. \quad (1.11)$$

Esta ecuación es válida, sin duda, para todo $x \neq 1$, y sigue siendo cierta cuando $x = 1$ si tomamos el límite indicado en el lado izquierdo.

¿Por qué es cierta (1.11)? Basta multiplicar ambos lados por $1 - x$ para eliminar fracciones. El resultado es

$$\begin{aligned} 1 - x^n &= (1 + x + x^2 + x^3 + \cdots + x^{n-1})(1 - x) \\ &= (1 + x + x^2 + \cdots + x^{n-1}) - (x + x^2 + x^3 + \cdots + x^n) \\ &= 1 - x^n, \end{aligned}$$

y la demostración queda concluida.

Ahora intenta este. ¿Cuál es el valor de la suma?

$$\sum_{j=0}^9 3^j?$$

Observa que estamos considerando el lado derecho de la ecuación (1.11) con $x = 3$. Por lo tanto, la respuesta es $\frac{3^{10}-1}{2}$.

Aquí hay algunas series más para mantener en tu zoológico. Un comentario entre paréntesis como ' $|x| < 1$ ' muestra el conjunto de valores de x para los cuales la serie converge.

$$\sum_{k=0}^{\infty} x^k = \frac{1}{1-x} \quad (|x| < 1) \quad (1.12)$$

$$e^x = \sum_{m=0}^{\infty} \frac{x^m}{m!} \quad (1.13)$$

$$\sin x = \sum_{r=0}^{\infty} \frac{(-1)^r x^{2r+1}}{(2r+1)!} \quad (1.14)$$

$$\cos x = \sum_{s=0}^{\infty} \frac{(-1)^s x^{2s}}{(2s)!} \quad (1.15)$$

$$\log \frac{1}{1-x} = \sum_{j=1}^{\infty} \frac{x^j}{j} \quad (|x| < 1) \quad (1.16)$$

¿Puedes encontrar una forma sencilla para la suma $1 + \log 2 + \frac{(\log 2)^2}{2!} + \frac{(\log 2)^3}{3!} + \dots$? (Pista: observa (1.13) y reemplaza x por $\log 2$.)

Aparte de simplemente sustituir valores de x en series conocidas, hay muchas otras maneras de usar series conocidas para expresar sumas en forma sencilla. Pensemos en la suma:

$$1 + 2 \cdot 2 + 3 \cdot 4 + 4 \cdot 8 + 5 \cdot 16 + \dots + N 2^{N-1}. \quad (1.17)$$

La serie geométrica finita (1.11) nos viene a la mente, pero (1.17) es un poco diferente debido a los multiplicadores $1, 2, 3, 4, \dots, N$.

El truco es el siguiente. Cuando te enfrentes a una serie que es similar, pero no idéntica, a una serie conocida, escribe la serie conocida como una ecuación, con la serie en un lado y su suma en el otro. Aunque la serie desconocida involucre un valor particular de x (en este caso $x = 2$), mantén la serie conocida con su variable sin restringir. Después, toma una herramienta apropiada que aplicarás a ambos lados de esa ecuación y cuyo resultado transformará la serie conocida en aquella cuya suma necesitas.

En este caso, dado que (1.17) nos recuerda a (1.11), empezaremos escribiendo nuevamente (1.11):

$$\frac{1 - x^n}{1 - x} = 1 + x + x^2 + \cdots + x^{n-1}. \quad (1.18)$$

Aún no sustituyas x por 2; acércate a la ecuación (1.18) con tu caja de herramientas y pregúntate qué clase de “cirugía” podrías hacer en ambos lados que sea útil para evaluar la serie desconocida (1.17).

Vamos a sacar de la caja la herramienta d/dx . Es decir, vamos a derivar (1.18). La razón para elegir la diferenciación es que añadirá los multiplicadores faltantes $1, 2, 3, \dots, N$ a (1.18). Después de derivar, obtenemos

$$1 + 2x + 3x^2 + 4x^3 + \cdots + (n-1)x^{n-2} = \frac{1 - nx^{n-1} + (n-1)x^n}{(1-x)^2}. \quad (1.19)$$

Ahora es fácil. Para evaluar la suma (1.17) basta sustituir $x = 2$ y $n = N+1$ en (1.19), con lo que, tras simplificar, obtenemos:

$$1 + 2 \cdot 2 + 3 \cdot 4 + 4 \cdot 8 + \cdots + N 2^{N-1} = 1 + (N-1)2^N. \quad (1.20)$$

Prueba ahora ésta:

$$\frac{1}{2 \cdot 3^2} + \frac{1}{3 \cdot 3^3} + \cdots \quad (1.21)$$

Si reescribimos la serie usando signos de suma, queda

$$\sum_{j=2}^{\infty} \frac{1}{j 3^j}.$$

Comparar con el “zoológico” de series revela un gran parecido con la especie (1.16). De hecho, si ponemos $x = \frac{1}{3}$ en (1.16), obtenemos

$$\sum_{j=1}^{\infty} \frac{1}{j 3^j} = \log\left(\frac{3}{2}\right). \quad (1.22)$$

La suma deseada (1.21) es el resultado de omitir el término con $j = 1$ de la ecuación (1.22), la cual muestra que la suma en (1.21) es igual a $\log(3/2) - 1/3$.

En general, supongamos que $f(x) = \sum a_n x^n$ es alguna serie conocida. Entonces:

$$\sum n a_n x^{n-1} = f'(x), \quad \sum n a_n x^n = x f'(x).$$

En otras palabras, si el coeficiente de orden n se multiplica por n , entonces la función cambia de f a $(x \frac{d}{dx}) f$. Si aplicamos la regla de nuevo, encontramos que multiplicar el coeficiente de orden n de una serie de potencias por n^2 cambia la suma de f a $(x \frac{d}{dx})^2 f$. Es decir:

$$\sum_{j=0}^{\infty} \frac{j^2 x^j}{j!} = \left(x \frac{d}{dx}\right) \left(x \frac{d}{dx}\right) e^x = \left(x \frac{d}{dx}\right) (x e^x) = (x^2 + x) e^x.$$

De manera similar, multiplicar el coeficiente de orden n de una serie de potencias por n^p hará que la suma de $f(x)$ cambie a $(x \frac{d}{dx})^p f(x)$, pero eso no es todo. ¿Qué ocurre si multiplicamos el coeficiente de x^n por, digamos, $3n^2 + 2n + 5$? Si la suma previa era $f(x)$, entonces ésta cambiará a

$$\{3(x \frac{d}{dx})^2 + 2(x \frac{d}{dx}) + 5\} f(x).$$

La suma

$$\sum_{j=0}^{\infty} (2j^2 + 5)x^j$$

es por tanto igual a

$$\{2(x \frac{d}{dx})^2 + 5\} \left\{ \frac{1}{1-x} \right\},$$

y tras realizar las diferenciaciones obtenemos la respuesta en la forma

$$\frac{7x^2 - 8x + 5}{(1-x)^3}.$$

Aquí está la regla general: si $P(x)$ es cualquier polinomio, entonces

$$\sum_j P(j) a_j x^j = P\left(x \frac{d}{dx}\right) \sum_j a_j x^j. \quad (1.23)$$

Ejercicios 1.3.1

1. Encuentre fórmulas simples y explícitas para las sumas de cada una de las siguientes series.

(a) $\sum_{j \geq 3} \frac{(\log 6)^j}{j!}$

(b) $\sum_{m > 1} \frac{2m+7}{5^m}$

(c) $\sum_{j=0}^{19} \frac{j}{2^j}$

(d) $1 - \frac{x}{2!} + \frac{x^2}{4!} - \frac{x^3}{6!} + \cdots$

(e) $1 - \frac{1}{3^2} + \frac{1}{3^4} - \frac{1}{3^6} + \dots$

(f) $\sum_{m=2}^{\infty} \frac{m^2 + 3m + 2}{m!}$

2. Explique por qué $\sum_{r \geq 0} \frac{(-1)^r \pi^{2r+1}}{(2r+1)!} = 0$.

3. Encuentre el coeficiente de t^n en la expansión en serie de cada una de las siguientes funciones en torno a $t = 0$:

(a) $(1 + t + t^2)e^t$

(b) $(3t - t^2) \sin t$

(c) $\frac{(t+1)^2}{(t-1)^2}$

4. Relaciones de Recurrencia

Una relación de recurrencia es una fórmula que nos permite calcular los términos de una sucesión uno tras otro, partiendo de uno o más valores iniciales. A continuación, un ejemplo sencillo. Supongamos que queremos encontrar una sucesión infinita de números $x_0, x_1, \dots$ mediante:

$$x_{n+1} = c x_n \quad (n \geq 0; x_0 = 1). \quad (1.24)$$

Esta relación indica que $x_1 = c x_0$, $x_2 = c x_1$, etc., y además que $x_0 = 1$. Por tanto, resulta que $x_1 = c$, $x_2 = c^2$, $\dots$, $x_n = c^n$, etc. Decimos que la solución de la relación de recurrencia (“ecuación en diferencias”) (1.24) está dada por $x_n = c^n$ para todo $n \geq 0$. La ecuación (1.24) es de primer orden porque cada nuevo término x_{n+1} se obtiene únicamente de x_n , sin involucrar términos anteriores.

$$x_{n+1} = x_n + x_{n-1} \quad (1.25)$$

Esta relación, de segundo orden, requiere dos valores iniciales para poder “arrancar”, pero no se especifican ni dichos valores ni el rango de n . En consecuencia, la sucesión no queda determinada de manera única. El planteamiento es análogo al de las ecuaciones diferenciales ordinarias: si se omiten condiciones iniciales o de contorno, las soluciones quedan determinadas solo hasta constantes arbitrarias.

Más allá de la sencilla (1.24), el siguiente nivel de dificultad ocurre cuando consideramos una relación de recurrencia de primer orden con un multiplicador variable, tal como

$$x_{n+1} = b_{n+1}x_n \quad (n \geq 0; x_0 \text{ dado}). \quad (1.26)$$

Ahora $\{b_1, b_2, \dots\}$ es una sucesión dada, y se nos está pidiendo encontrar la sucesión desconocida $\{x_1, x_2, \dots\}$.

En un caso sencillo como este, podemos escribir los primeros valores de x_n y luego adivinar la respuesta. Encontramos, sucesivamente, que $x_1 = b_1x_0$, luego $x_2 = b_2x_1 = b_2b_1x_0$ y $x_3 = b_3x_2 = b_3b_2b_1x_0$, etc. En este punto podemos conjeturar que la solución es

$$x_n = x_0 \prod_{i=1}^n b_i \quad (n = 0, 1, 2, \dots). \quad (1.27)$$

Puesto que eso no fue lo suficientemente difícil, subiremos la apuesta un paso más. Supongamos que queremos resolver la relación de recurrencia de primer orden inhomogénea (porque $x_n = 0$ para todo n no es solución):

$$x_{n+1} = b_{n+1}x_n + c_{n+1} \quad (n \geq 0; x_0 \text{ dado}). \quad (1.28)$$

Ahora se nos dan dos sucesiones $b_1, b_2, \dots$ y $c_1, c_2, \dots$, y queremos encontrar los x_n . Supongamos que seguimos la estrategia que hasta ahora ha ganado el juego, es decir, escribir los primeros x_n e intentar adivinar el patrón. Entonces encontraríamos que $x_1 = b_1x_0 + c_1$, $x_2 = b_2b_1x_0 + b_2c_1 + c_2$, y probablemente nos cansaríamos rápidamente.

Aquí hay un enfoque algo más ordenado para (1.28). Aunque ningún enfoque evitará la forma desagradable de la respuesta general, el que estamos a punto de describir al menos proporciona un método mucho más sencillo que la estrategia de conjetura, para muchos ejemplos que aparecen en la práctica. En este libro nos encontraremos con varias ecuaciones del tipo de (1.28), por lo que un método unificado será una ventaja definitiva.

El primer paso es definir una nueva función desconocida de la siguiente manera. Sea

$$x_n = b_1b_2 \cdots b_n y_n \quad (n \geq 1; x_0 = y_0) \quad (1.29)$$

la que define una nueva sucesión desconocida $y_1, y_2, \dots$. Ahora sustituimos x_n en (1.28), obteniendo

$$b_1b_2 \cdots b_{n+1} y_{n+1} = b_{n+1}b_1b_2 \cdots b_n y_n + c_{n+1}.$$

Notamos que los coeficientes de y_{n+1} y de y_n son los mismos, y por lo tanto dividimos ambos lados por dicho coeficiente. El resultado es la ecuación

$$y_{n+1} = y_n + d_{n+1} \quad (n \geq 0; y_0 \text{ dado}). \quad (1.30)$$

Observemos que hemos escrito $d_{n+1} = c_{n+1}/(b_1 \cdots b_{n+1})$. Obsérvese que los d son conocidos.

Aún no hemos resuelto la relación de recurrencia; sólo la hemos transformado en una nueva función desconocida que satisface una recurrencia más sencilla, (??). La solución de (??) es bastante simple, pues indica que cada y se obtiene de su predecesor añadiendo el siguiente de los d . De esto se deduce que

$$y_n = y_0 + \sum_{j=1}^n d_j \quad (n \geq 0).$$

Podemos ahora emplear (??) para deshacer el cambio de variables y volver a las incógnitas originales $x_0, x_1, \dots$; obtenemos

$$x_n = (b_1 b_2 \cdots b_n) \left\{ x_0 + \sum_{j=1}^n d_j \right\} \quad (n \geq 1). \quad (1.31)$$

No se recomienda al lector memorizar la solución recién obtenida; sí se recomienda dominar el método mediante el cual se halló. Este consiste en:

- (a) realizar un cambio de variables que conduzca a una nueva recurrencia de la forma (??);
- (b) resolver dicha recurrencia por sumación, y
- (c) regresar a las incógnitas originales.

Como ejemplo, consideremos la ecuación de primer orden

$$x_{n+1} = 3x_n + n \quad (n \geq 0; x_0 = 0). \quad (1.32)$$

El cambio de variable “ganador” dado por (??) es $x_n = 3^n y_n$. Sustituyendo en (??) y simplificando se obtiene

$$y_{n+1} = y_n + \frac{n}{3^{n+1}} \quad (n \geq 0; y_0 = 0).$$

Por sumación,

$$y_n = \sum_{j=1}^{n-1} \frac{j}{3^{j+1}} \quad (n \geq 0).$$

Finalmente, como $x_n = 3^n y_n$, la solución de (??) queda

$$x_n = 3^n \sum_{j=1}^{n-1} \frac{j}{3^{j+1}} \quad (n \geq 0). \quad (1.33)$$

Esta es una respuesta bastante explícita, pero de hecho la sumatoria puede eliminarse completamente mediante el mismo método que utilizaste para resolver el Ejercicio 1(c) de la Sección 1.3 (¡inténtalo!).

Con esto hemos cubierto bastante bien las relaciones de recurrencia de primer orden de la forma $x_{n+1} = b_{n+1}x_n + c_{n+1}$, y es momento de pasar a las relaciones de recurrencia lineales de segundo orden *homogéneas* con coeficientes constantes. Estas son de la forma:

$$x_{n+1} = ax_n + bx_{n-1} \quad (n \geq 1; x_0 \text{ y } x_1 \text{ dados}). \quad (1.34)$$

Si recordamos las ecuaciones diferenciales de segundo orden con coeficientes constantes, sabemos que siempre poseen soluciones de la forma $y(t) = e^{\alpha t}$, donde α es constante. Por lo tanto, el camino hacia la solución de tal ecuación diferencial comienza proponiendo una solución de esa forma y determinando qué valor o valores toma α .

De manera análoga, la ecuación (??) sugiere ensayar una solución de la forma $x_n = \alpha^n$. Al sustituir $x_n = \alpha^n$ en (??) y cancelar el factor común α^{n-1} , obtenemos una ecuación cuadrática para α , a saber

$$\alpha^2 = a\alpha + b. \quad (1.35)$$

‘Usualmente’ esta ecuación cuadrática tiene dos raíces distintas, digamos α_+ y α_- , y entonces la solución general de (??) será

$$x_n = c_1\alpha_+^n + c_2\alpha_-^n \quad (n = 0, 1, 2, \dots). \quad (1.36)$$

Las constantes c_1 y c_2 se determinan de modo que x_0 y x_1 tomen los valores asignados.

Ejemplo 1.2. La recurrencia para los números de Fibonacci es:

$$F_{n+1} = F_n + F_{n-1} \quad (n \geq 1; F_0 = 0; F_1 = 1). \quad (1.37)$$

Siguiendo la receta descrita arriba, buscamos una solución de la forma $F_n = \alpha^n$. Al sustituir en (??) y cancelar factores comunes obtenemos que la ecuación cuadrática para α es, en este caso, $\alpha^2 = \alpha + 1$.

Si denotamos las dos raíces por $\alpha_+ = (1 + \sqrt{5})/2$ y $\alpha_- = (1 - \sqrt{5})/2$, entonces ya contamos con la solución general de la recurrencia de Fibonacci, y ésta tiene la forma (??). Falta determinar las constantes c_1, c_2 a partir de las condiciones iniciales $F_0 = 0, F_1 = 1$.

A partir de la forma de la solución general se tiene $F_0 = 0 = c_1 + c_2$ y $F_1 = 1 = c_1\alpha_+ + c_2\alpha_-$. Si resolvemos estas dos ecuaciones para las incógnitas c_1, c_2 y sustituimos estos valores de las constantes en la expresión...

A partir de la forma de la solución general obtenemos una fórmula explícita para el n -ésimo número de Fibonacci:

$$F_n = \frac{1}{\sqrt{5}} \left[\left(\frac{1 + \sqrt{5}}{2} \right)^n - \left(\frac{1 - \sqrt{5}}{2} \right)^n \right] \quad (n = 0, 1, \dots). \quad (1.38)$$

Los números de Fibonacci son, de hecho, $0, 1, 1, 2, 3, 5, 8, 13, 21, 34, \dots$. Ni siquiera es evidente que la fórmula (1.38) produzca valores enteros para los F_n . Se invita al lector a verificar que, en efecto, la fórmula reproduce correctamente los primeros términos.

Para ejercitar nuestras habilidades recién adquiridas en asintótica, observemos que, dado que $\frac{1+\sqrt{5}}{2} > 1$ y $\left| \frac{1-\sqrt{5}}{2} \right| < 1$, se sigue que, cuando n es grande,

$$F_n \sim \frac{1}{\sqrt{5}} \left(\frac{1 + \sqrt{5}}{2} \right)^{n+1}.$$

El método de buscar una solución en la forma α^n requiere un tratamiento especial cuando la ecuación cuadrática asociada $\alpha^2 = a\alpha + b$ posee una raíz doble. En tal caso existen dos soluciones independientes, α^n y $n\alpha^n$, de modo que la solución general adopta la forma $\alpha^n(c_1 + c_2 n)$.

Ejemplo 1.3. Consideremos la recurrencia

$$x_{n+1} = 2x_n - x_{n-1} \quad (n \geq 1; x_0 = 1; x_1 = 5). \quad (1.39)$$

Si ensayamos una solución $x_n = \alpha^n$, encontramos que α satisface la ecuación cuadrática $\alpha^2 = 2\alpha - 1$. Así pues, las “dos” raíces son 1 y 1. La solución general resulta

$$x_n = 1^n(c_1 + n c_2) = c_1 + c_2 n.$$

Aplicando las condiciones iniciales,

$$x_0 = 1 = c_1, \quad x_1 = 5 = c_1 + c_2,$$

se obtiene $c_1 = 1$, $c_2 = 4$; por lo tanto, la solución completa de la recurrencia (1.39) es

$$x_n = 4n + 1.$$

Ahora examinemos una desigualdad recursiva del tipo

$$x_{n+1} \leq x_n + x_{n-1} + n^2 \quad (n \geq 1; x_0 = 0; x_1 = 0). \quad (1.40)$$

La cuestión es: ¿qué restricción impone (1.40) al crecimiento de la sucesión $\{x_n\}$?

Por analogía con el caso de ecuaciones en diferencias con coeficientes constantes, lo que se intenta aquí es suponer

$$x_n \leq K\alpha^n.$$

Supongamos que esto es cierto para todo $n = 0, 1, 2, \dots, N$. Entonces, a partir de (??) con $n = N$ se deduce

$$x_{N+1} \leq K\alpha^N + K\alpha^{N-1} + N^2.$$

Sea c la raíz real positiva de $c^2 = c + 1$ y supongamos $\alpha > c$. Puesto que $\alpha^2 > \alpha + 1$, podemos escribir $\alpha^2 - \alpha - 1 = t$, con $t > 0$. Se sigue que

$$\begin{aligned} x_{N+1} &\leq K\alpha^{N-1}(1 + \alpha) + N^2 \\ &= K\alpha^{N-1}(\alpha^2 - t) + N^2 \\ &= K\alpha^{N+1} - (tK\alpha^{N-1} - N^2). \end{aligned} \tag{1.41}$$

Para asegurar que $x_{N+1} < K\alpha^{N+1}$ necesitamos que $tK\alpha^{N-1} > N^2$. Por lo tanto, basta elegir

$$K > \max_{N \geq 2} \frac{N^2}{t\alpha^{N-1}}, \tag{1.42}$$

cuyo valor es claramente finito, y el paso inductivo queda verificado.

Concluimos que (??) implica que, para todo $\varepsilon > 0$ fijo,

$$x_n = O((c + \varepsilon)^n), \quad c = \frac{1 + \sqrt{5}}{2}.$$

El mismo razonamiento se aplica a la situación general recogida en el

Teorema 1.10. Sea $\{x_n\}$ una sucesión que satisface la desigualdad recursiva

$$x_{n+1} \leq b_0x_n + b_1x_{n-1} + \dots + b_px_{n-p} + G(n) \quad (n \geq p),$$

donde $b_i \geq 0$ ($\forall i$) y $\sum b_i > 1$. Sea c la raíz real positiva³ de la ecuación

$$c^{p+1} = b_0c^p + \dots + b_p.$$

Si además $G(n) = o(c^n)$, entonces, para todo $\varepsilon > 0$ fijo,

$$x_n = O((c + \varepsilon)^n).$$

Demostración. Fijemos $\varepsilon > 0$ y tomemos $\alpha = c + \varepsilon$. Como $\alpha > c$, definamos

$$t = \alpha^{p+1} - b_0\alpha^p - \dots - b_p,$$

³Véase el Ejercicio 10 de la Sección 1.4.1.

de modo que $t > 0$. Sea

$$K = \max\left\{|x_0|, \frac{|x_1|}{\alpha}, \dots, \frac{|x_p|}{\alpha^p}, \max_{n \geq p} \frac{G(n)}{t \alpha^{n-p}}\right\}.$$

Con esta elección de K la inducción procede exactamente como antes, y el enunciado queda probado.

Entonces K es finito y, claramente, $|x_j| \leq K\alpha^j$ para $j \leq p$. Afirmamos que $|x_n| \leq K\alpha^n$ para todo n , lo que completará la demostración. En efecto, si la afirmación es cierta para $0, 1, 2, \dots, n$, entonces

$$\begin{aligned}
 |x_{n+1}| &\leq b_0|x_n| + \dots + b_p|x_{n-p}| + G(n) \\
 &\leq b_0K\alpha^n + \dots + b_pK\alpha^{n-p} + G(n) \\
 &= K\alpha^{n-p}\{b_0\alpha^p + \dots + b_p\} + G(n) \\
 &= K\alpha^{n-p}\{\alpha^{p+1} - t\} + G(n) \\
 &= K\alpha^{n+1} - \{tK\alpha^{n-p} - G(n)\} \\
 &\leq K\alpha^{n+1}.
 \end{aligned}$$

□

1.4.1 Ejercicios

1. Resuelva las siguientes relaciones de recurrencia:

- (a) $x_{n+1} = x_n + 3 \quad (n \geq 0; x_0 = 1)$
- (b) $x_{n+1} = \frac{x_n}{3} + 2 \quad (n \geq 0; x_0 = 0)$
- (c) $x_{n+1} = 2(n+1)x_n \quad (n \geq 0; x_0 = 1)$
- (d) $x_{n+1} = \frac{n+1}{n}x_n + n + 1 \quad (n \geq 1; x_1 = 5)$
- (e) $x_{n+1} = x_n + x_{n-1} \quad (n \geq 1; x_0 = 0; x_1 = 3)$
- (f) $x_{n+1} = 3x_n - 2x_{n-1} \quad (n \geq 1; x_0 = 1; x_1 = 3)$
- (g) $x_{n+1} = 4x_n - 4x_{n-1} \quad (n \geq 1; x_0 = 1; x_1 = \xi)$

- 2. Encuentre x_1 si la sucesión $\{x_n\}$ satisface la recurrencia de Fibonacci y, además, $x_0 = 1$ y $x_n = o(1)$ cuando $n \rightarrow \infty$.
- 3. Sea x_n el número promedio de ceros finales en las expansiones binarias de los enteros $0, 1, 2, \dots, 2^n - 1$. Calcule x_n y evalúe $\lim_{n \rightarrow \infty} x_n$.
- 4. ¿Para qué valores de a y b se cumple que, sin importar los valores iniciales x_0, x_1 , la solución de $x_{n+1} = ax_n + bx_{n-1} \quad (n \geq 1)$ satisface $x_n = o(1)$ cuando $n \rightarrow \infty$?
- 5. Suponga $x_0 = 0, x_1 = 1$ y, para todo $n \geq 2, x_{n+1} \leq x_n + x_{n-1}$. ¿Es cierto que $\forall n : x_n \leq F_n$? Demuestre su respuesta.

1. Generalice el resultado del Ejercicio 5 del modo siguiente. Suponga que $x_0 = y_0$ y $x_1 = y_1$, donde $y_{n+1} = ay_n + by_{n-1}$ ($\forall n \geq 1$). Si además $x_{n+1} \leq ax_n + bx_{n-1}$ ($\forall n \geq 1$), ¿podemos concluir que $\forall n : x_n \leq y_n$? Si la respuesta es negativa, describa condiciones sobre a y b bajo las cuales la conclusión sea cierta.
2. Encuentre el comportamiento asintótico, en la forma $x_n \sim? (n \rightarrow \infty)$, del lado derecho de (1.33).
3. Escriba una demostración completa del Teorema 1.3.
4. En el Teorema 1.3 aparece la frase «... la raíz real positiva de ...». Demuestre que dicha frase está justificada, es decir, que la ecuación indicada siempre posee exactamente una raíz real positiva. ¿Qué propiedades especiales de esa ecuación utilizó en su prueba?

1.5 Conteo

Para un entero positivo n , consideremos el conjunto $\{1, 2, \dots, n\}$, que denotaremos por $[n]$. Queremos analizar cuántos subconjuntos de diversos tipos posee. Por ejemplo, la lista completa de subconjuntos de $[2]$ es $\emptyset$, $\{1\}$, $\{2\}$ y $\{1, 2\}$, es decir, 4 subconjuntos.

Afirmamos que $[n]$ tiene exactamente 2^n subconjuntos. En efecto, podemos construir cada subconjunto decidiendo, para cada uno de los n elementos, si lo incluimos o no. Cada decisión admite dos alternativas, de modo que el total de decisiones se puede tomar de 2^n formas; ese es, por tanto, el número de subconjuntos de un conjunto con n elementos.

Supongamos ahora que tenemos n objetos distintos y deseamos ordenarlos en una secuencia. ¿Cuántas permutaciones existen? Para el primer lugar podemos elegir cualquiera de los n objetos; para el segundo, cualquiera de los $n - 1$ restantes, dando $n(n - 1)$ maneras de fijar los dos primeros lugares. Para el tercero quedan $n - 2$ opciones, y así sucesivamente, de modo que el número total de secuencias es

$$n(n - 1)(n - 2) \cdots 3 \cdot 2 \cdot 1 = n!.$$

De los 2^n subconjuntos de $[n]$, ¿cuántos contienen exactamente k elementos? El número de elementos de un conjunto se denomina *cardinalidad* y se denota por $|S|$. Por ejemplo, $|[6]| = 6$. Un conjunto de cardinalidad k se llama *k-conjunto*, y un subconjunto de cardinalidad k es, naturalmente, un *k-subconjunto*. La pregunta es: ¿cuántos subconjuntos S de $[n]$ satisfacen $|S| = k$?

Podemos construir k -subconjuntos S de $[n]$ (escrito $S \subseteq [n]$) de la siguiente manera. Elegimos un elemento a_1 (n posibles elecciones). De los $n - 1$ elementos restantes, elegimos uno ($n - 1$ posibles elecciones), etc., hasta que se haya escogido una secuencia de k elementos diferentes. Obviamente hubo

$$n(n-1)(n-2)\cdots(n-k+1)$$

formas en las que podríamos haber elegido esa secuencia, por lo que el número de formas de elegir una secuencia *ordenada* de k elementos de $[n]$ es:

$$n(n-1)(n-2)\cdots(n-k+1) = \frac{n!}{(n-k)!}.$$

Pero hay más secuencias de k elementos que k -subconjuntos, porque cualquier k -subconjunto particular S corresponderá a $k!$ secuencias ordenadas diferentes, es decir, todas las posibles reordenaciones de los elementos del subconjunto. Por lo tanto, el número de k -subconjuntos de $[n]$ es igual al número de k -secuencias dividido por $k!$. En otras palabras, hay exactamente $\frac{n!}{k!(n-k)!}$ k -subconjuntos de un conjunto de n objetos.

Las cantidades $\frac{n!}{k!(n-k)!}$ son los famosos *coeficientes binomiales*, y se denotan por:

$$\binom{n}{k} = \frac{n!}{k!(n-k)!} \quad (n \geq k \geq 0). \quad (1.43)$$

Algunos de sus valores especiales son:

$$\binom{n}{0} = 1 \quad (\forall n \geq 0), \quad \binom{n}{1} = n \quad (\forall n \geq 0),$$

$$\binom{n}{2} = \frac{n(n-1)}{2} \quad (\forall n \geq 0), \quad \binom{n}{n} = 1 \quad (\forall n \geq 0).$$

Es conveniente definir $\binom{n}{k}$ como 0 si $k < 0$ o si $k > n \geq 0$.

Podemos resumir los desarrollos hasta ahora con:

Teorema 2. *Para cada $n \geq 0$, un conjunto de n objetos tiene exactamente 2^n subconjuntos, y de estos, exactamente $\binom{n}{k}$ tienen cardinalidad k ($\forall k = 0, 1, \dots, n$). Hay exactamente $n!$ secuencias diferentes que pueden formarse a partir de un conjunto de n objetos distintos.*

Dado que cada subconjunto de $[n]$ tiene alguna cardinalidad, se sigue que:

$$\sum_{k=0}^n \binom{n}{k} = 2^n \quad (n = 0, 1, 2, \dots). \quad (1.44)$$

En vista de la convención que adoptamos, podríamos haber escrito (1.44) como

$$\sum_k \binom{n}{k} = 2^n \quad (n \geq 0),$$

sin restricción sobre el rango de la sumatoria.

[illegible]

Cuadro 1: Triángulo de Pascal.

índice k . Entonces se habría entendido que el rango de k va de $-\infty$ a ∞ , y que el coeficiente binomial $\binom{n}{k}$ se anula a menos que $0 \leq k \leq n$. En la Tabla 1.1, mostramos los valores de algunos de los coeficientes binomiales $\binom{n}{k}$. Las filas de la tabla se consideran etiquetadas ‘ $n = 0$ ’, ‘ $n = 1$ ’, etc., y las entradas dentro de cada fila se refieren, sucesivamente, a $k = 0, 1, \dots, n$. La tabla se denomina “triángulo de Pascal”.

He aquí algunos hechos sobre los coeficientes binomiales:

- (a) Cada fila del triángulo de Pascal es simétrica respecto del centro. Es decir:

$$\binom{n}{k} = \binom{n}{n-k} \quad (0 \leq k \leq n; n \geq 0).$$

- (b) La suma de las entradas en la n -ésima fila del triángulo de Pascal es 2^n .
- (c) Cada entrada es igual a la suma de las dos entradas que están inmediatamente encima de ella en el triángulo.

La demostración de (c) anterior puede ser interesante. Lo que afirma sobre los coeficientes binomiales es que:

$$\binom{n}{k} = \binom{n-1}{k-1} + \binom{n-1}{k} \quad ((n, k) \neq (0, 0)). \quad (1.45)$$

Hay (al menos) dos maneras de demostrar (1.45). El enfoque de fuerza bruta consistiría en expandir cada uno de los tres coeficientes binomiales que aparece en (1.45), usando la definición (1.43) en términos de factoriales, y luego cancelar factores comunes para completar la demostración.

Eso funcionaría (inténtelo), pero aquí hay otra manera. Contemple (esta demostración es por contemplación) la totalidad de los k -subconjuntos de $[n]$. Su número está en el lado izquierdo de (1.45). Sepárelos en dos montones: aquellos

k -subconjuntos que contienen ‘1’ y los que no. Si un k -subconjunto de $[n]$ contiene ‘1’, entonces sus $k - 1$ elementos restantes pueden elegirse de $\binom{n-1}{k-1}$ maneras, y eso explica el primer término en el lado derecho de (1.45). Si un k -subconjunto no contiene ‘1’, entonces sus k elementos se eligen todos de $[n - 1]$, y eso completa la demostración de (1.45).

El teorema binomial afirma que $\forall n \geq 0$ tenemos

$$(1 + x)^n = \sum_{k=0}^n \binom{n}{k} x^k. \quad (1.46)$$

Demostración. Por inducción sobre n . La ecuación (1.46) es claramente cierta cuando $n = 0$, y si es cierta para algún n , entonces multiplicamos ambos lados de (1.46) por $(1 + x)$ para obtener

$$\begin{aligned} (1 + x)^{n+1} &= \sum_k \binom{n}{k} x^k + \sum_k \binom{n}{k} x^{k+1} \\ &= \sum_k \binom{n}{k} x^k + \sum_k \binom{n}{k-1} x^k \\ &= \sum_k \left(\binom{n}{k} + \binom{n}{k-1} \right) x^k \\ &= \sum_k \binom{n+1}{k} x^k, \end{aligned}$$

lo que completa la demostración.

Ahora preguntémonos qué tan grandes son los coeficientes binomiales, como un ejercicio de análisis asintótico. Nos referiremos a los coeficientes en la fila n del triángulo de Pascal, es decir, a $\binom{n}{0}, \binom{n}{1}, \dots, \binom{n}{n}$ como los coeficientes de orden n . Entonces, por (1.44) (o por (1.46) con $x = 1$), la suma de todos los coeficientes de orden n es 2^n . También es bastante evidente, a partir de una inspección de la Tabla 1.1, que el(los) mayor(es) de los coeficientes de orden n está(n) en el medio.

Más precisamente, si n es impar, entonces los coeficientes más grandes de orden n son $\binom{n}{(n-1)/2}$ y $\binom{n}{(n+1)/2}$, mientras que si n es par, el mayor es de forma única $\binom{n}{n/2}$. Será importante, en algunas de las aplicaciones a algoritmos más adelante en este libro, poder seleccionar el término más grande de una secuencia de este tipo, así que veamos cómo podríamos probar que los coeficientes más grandes son los indicados arriba.

Para n fijo, calcularemos la razón del $(k + 1)$ -ésimo coeficiente de orden n respecto del k -ésimo. Veremos entonces que la razón es mayor que 1 si $k < (n - 1)/2$ y es < 1 si $k > (n - 1)/2$. Eso, por supuesto, implicará que el $(k + 1)$ -ésimo coeficiente es mayor que el k -ésimo para tales k , y por lo tanto que el(los) más grande(s) debe(n) estar en el medio.

La razón es

$$\frac{\binom{n}{k+1}}{\binom{n}{k}} = \frac{n!/((k+1)!(n-k-1)!)}{n!/(k!(n-k)!)} = \frac{k!(n-k)!}{(k+1)!(n-k-1)!} = \frac{n-k}{k+1},$$

y es > 1 si y sólo si $k < (n - 1)/2$, como se afirmó.

Bien, los coeficientes más grandes están en el medio, pero ¿qué tan grandes son? Supongamos que n es par, sólo para mantener las cosas simples. Entonces el mayor coeficiente binomial de orden n es

$$\binom{n}{n/2} = \frac{n!}{((n/2)!)^2} \sim \frac{\sqrt{2\pi n} (n/e)^n}{(\sqrt{\pi n} (n/(2e))^{n/2})^2} = \sqrt{\frac{2}{n\pi}} 2^n, \quad (1.47)$$

donde hemos usado la fórmula de Stirling (1.10).

La ecuación (1.47) muestra que el coeficiente binomial más grande por sí solo representa una fracción muy sustancial de la suma de todos los coeficientes de orden n . De hecho, la suma de todos ellos es 2^n , y el mayor es $\sim \sqrt{2/(n\pi)} 2^n$. Cuando n es grande, por lo tanto, el coeficiente más grande aporta una fracción $\sim \sqrt{2/(n\pi)}$ del total.

Si pensamos en términos de los subconjuntos que estos coeficientes cuentan, lo que veremos es que una gran fracción de todos los subconjuntos de un n -conjunto tienen cardinalidad $n/2$; de hecho, una fracción $\Theta(n^{-0.5})$ de ellos. Este tipo de pensamiento probabilístico puede ser muy útil en el diseño y análisis de algoritmos. Si estamos diseñando un algoritmo que trabaja con subconjuntos de $[n]$, por ejemplo, debemos reconocer que un gran porcentaje de los casos de uso para ese algoritmo tendrán cardinalidades cercanas a $n/2$, y hacer todo lo posible por lograr que el algoritmo sea rápido para tales subconjuntos, aun a costa de posiblemente hacerlo más lento en subconjuntos cuyas cardinalidades sean muy pequeñas o muy grandes.

4.0.1. Ejercicios

1. ¿Cuántos subconjuntos de cardinalidad par tiene $[n]$?
2. Observando que $(1+x)^a(1+x)^b = (1+x)^{a+b}$, pruebe que la suma de los cuadrados de todos los coeficientes binomiales de orden n es $\binom{2n}{n}$.
3. Evalúe las siguientes sumas en forma simple.

a) $\sum_{j=0}^n j \binom{n}{j}$

b) $\sum_{j=3}^n \binom{n}{j} 5^j$

c) $\sum_{j=0}^n (j+1) 3^{j+1}$

4. Halle, por aplicación directa del teorema de Taylor, el desarrollo en serie de potencias de $f(x) = \frac{1}{(1-x)^{m+1}}$ alrededor del origen. Expresé los coeficientes como ciertos coeficientes binomiales.
5. Complete las siguientes aproximaciones asintóticas ($\sim$).

a) $\binom{2n}{n} \sim ?$

b) $\binom{n}{\lfloor \log_2 n \rfloor} \sim ?$

c) $\binom{n}{\lfloor \theta n \rfloor} \sim ?$

d) $\binom{n^2}{n} \sim ?$

6. ¿Cuántos pares ordenados de elementos distintos de $[n]$ hay?
7. ¿Cuál de los números $\{2^j \binom{n}{j}\}_{j=0}^n$ es el mayor?

5. Grafos

Un grafo es una colección de vértices, ciertos pares no ordenados de los cuales se llaman sus aristas. Para describir un grafo particular, primero decimos cuáles son sus vértices y luego decimos cuáles pares de vértices son sus aristas. El conjunto de vértices de un grafo G se denota por $V(G)$, y su conjunto de aristas por $E(G)$.

Si v y w son vértices de un grafo G , y si (v, w) es una arista de G , entonces decimos que los vértices v, w son adyacentes en G .

Considere el grafo G cuyo conjunto de vértices es $\{1, 2, 3, 4, 5\}$ y cuyas aristas son el conjunto de pares $(1, 2)$, $(2, 3)$, $(3, 4)$, $(4, 5)$, $(1, 5)$. Este es un grafo de 5 vértices y 5 aristas. Una manera agradable de presentar un grafo a una audiencia es dibujar una figura de él, en lugar de simplemente listar los pares de vértices que son sus aristas. Para dibujar una figura de un grafo, primero haríamos un punto por cada vértice, y luego trazariamos un arco entre dos vértices v y w si y sólo si (v, w) es una arista del grafo del que estamos hablando. El grafo

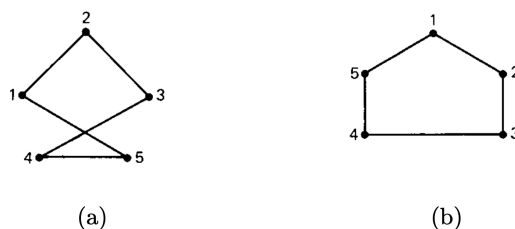


Figura 5: Figura 1.3.

El G de 5 vértices y 5 aristas que listamos arriba puede dibujarse como se muestra en la Figura 1.3(a). También podría dibujarse como se muestra en la Figura 1.3(b). Ambos son el mismo grafo. Solo las figuras son diferentes, pero las figuras no son «realmente» el grafo; el grafo es la lista de vértices y la lista de aristas. Las figuras nos ayudan a visualizar y recordar el grafo, pero nada más.

El número de aristas que contienen («son incidentes con») un vértice particular v de un grafo G se llama el *grado* de ese vértice, y suele denotarse por $\rho(v)$. Si sumamos los grados de cada vértice v de G , habremos contado exactamente dos contribuciones de cada arista de G , una en cada uno de sus extremos. Por lo tanto, para todo grafo G se tiene:

$$\sum_{v \in V(G)} \rho(v) = 2|E(G)|. \quad (1.48)$$

Como el lado derecho es un número par, debe haber un número par de números impares en el lado izquierdo de (1.48). Hemos probado, por tanto, que todo grafo tiene un número par de vértices cuyos grados son impares.² En la Figura 1.3 los grados de los vértices son 2, 2, 2, 2, 2 y la suma de los grados es $10 = 2|E(G)|$.

A continuación, vamos a definir una serie de conceptos de teoría de grafos que serán necesarios en capítulos posteriores. Ahora se definirá un número bastante grande de términos, en un espacio más bien breve. No intentes asimilarlos todos ahora; léelos y vuélvelos a mirar cuando los conceptos se utilicen realmente, más adelante.

Un *camino* P en un grafo G es un recorrido desde un vértice de G hasta otro, donde en cada paso el recorrido usa una arista del grafo. Más formalmente, es una secuencia $v_1, v_2, \dots, v_k$ de vértices de G tal que $\forall i = 1, k-1 : (v_i, v_{i+1}) \in E(G)$.

²¿Te diste cuenta de que el número de personas que estrecharon la mano un número impar de veces ayer es un número par de personas?

Un grafo es *conexo* si existe un camino entre cada par de sus vértices. Un camino P es *simple* si sus vértices son todos distintos, *hamiltoniano* si es simple y visita cada vértice de G exactamente una vez, y *euleriano* si utiliza cada arista de G exactamente una vez.

Un *subgrafo* de un grafo G es un subconjunto S de sus vértices junto con un subconjunto únicamente de aquellas aristas de G cuyos dos extremos yacen en S . Un *subgrafo inducido* de G es un subconjunto S de los vértices de G junto con todas las aristas de G cuyos dos extremos yacen en S . En tal caso hablaríamos del “subgrafo inducido por S ”.

En un grafo G , podemos definir una relación de equivalencia sobre los vértices como sigue. Diremos que v y w son equivalentes si existe un camino de G que los une. Sea S una de las clases de equivalencia de vértices de G bajo esta relación. El subgrafo de G que S induce se denomina un *componente conexo* del grafo G . Un grafo es conexo si y solo si tiene exactamente un componente conexo.

Un *ciclo* es un camino cerrado, es decir, uno en el que $v_k = v_1$. Un ciclo es un *circuito* si v_1 es el único vértice repetido en él. Podemos decir que un circuito es un ciclo simple. Hablamos de circuitos hamiltonianos y eulerianos de G como circuitos de G que visitan, respectivamente, cada vértice, o cada arista, de un grafo G .

Sin embargo, existe un abismo entre los caminos eulerianos y los hamiltonianos. Dado un grafo G , gracias al siguiente elegante teorema de Euler, es bastante fácil decidir si G tiene o no un camino euleriano. De hecho, el teorema también se aplica a los *multígrafos*, que son grafos salvo que permiten tener varias aristas diferentes que unan el mismo par de vértices.

Teorema 1.12. Un (multi)grafo tiene un circuito euleriano (respectivamente, un camino) si y solo si es conexo y no tiene (respectivamente, tiene exactamente dos) vértices de grado impar.

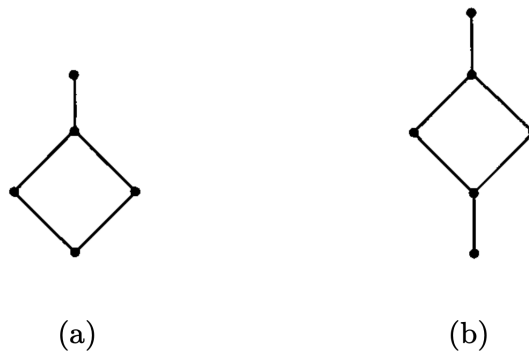


Figura 6: No todo grafo tiene un camino hamiltoniano. El grafo de la Figura 1.4(a) tiene uno y el grafo de la Figura 1.4(b) no lo tiene.

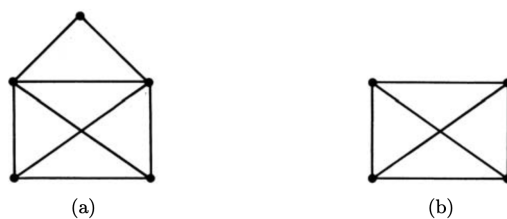


Figura 7: Del mismo modo, no todo grafo tiene un camino euleriano. El grafo en la Figura 1.5(a) tiene uno y el grafo en la Figura 1.5(b) no.

Demostración. Sea G un multigrafo conexo en el que todo vértice tiene grado par. Encontraremos un circuito euleriano en G . La demostración para caminos eulerianos será similar y se omite.

La demostración es por inducción sobre el número de aristas de G , y el resultado es claramente cierto si G tiene solo una arista.

Por lo tanto, supongamos que el teorema es verdadero para todos esos multigrafos con menos de m aristas, y que G tiene m aristas. Construiremos un circuito euleriano de G .

Comenzamos en algún vértice v y recorremos alguna arista hasta un vértice w . En general, al llegar a un vértice u , salimos de u por una arista que aún no se haya usado, llegando a un nuevo vértice, etc. El proceso se detiene cuando llegamos por primera vez a un vértice v_0 tal que todas las aristas incidentes con v_0 ya han sido recorridas, de modo que no hay salida.

Afirmamos que $v_0 = v$, es decir, hemos vuelto al punto de partida. En efecto, si no fuera así, entonces habríamos llegado a v_0 una vez más de las que salimos de él, usando cada vez una arista nueva y, al final, no quedando ninguna arista disponible. Por lo tanto, había un número impar de aristas de G incidentes con v_0 , una contradicción.

Por consiguiente, efectivamente estamos de vuelta en el punto de inicio cuando la caminata termina. Sea W la secuencia de aristas que hemos recorrido hasta ahora. Si W incluye todas las aristas de G , entonces hemos encontrado un circuito euleriano y hemos terminado.

En caso contrario, hay aristas de G que no están en W . Eliminemos de G todas las aristas de W , obteniendo así un multigrafo (posiblemente desconectado) G' . Sean $C_1, \dots, C_k$ las componentes conexas de G' . Cada una de ellas tiene solo vértices de grado par porque eso era cierto de G y de la caminata W que sustrajimos de G . Como cada C_i tiene menos aristas que G , por inducción hay un circuito euleriano en cada una de las componentes conexas de G' .

Los entrelazaremos todos para construir tal circuito para el propio G .

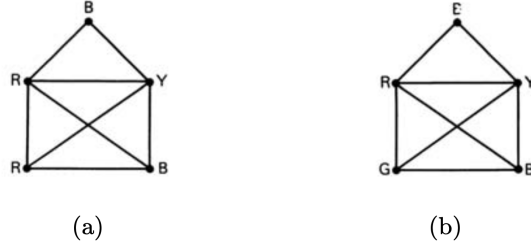


Figura 8: Figura 1.6.

Comience en el mismo v y camine a lo largo de 0 o más aristas de W hasta llegar, por primera vez, a un vértice q de la componente C_1 . Esto ciertamente ocurrirá porque G es conexo. Luego siga el recorrido de Euler de las aristas de C_1 , lo cual lo devolverá al vértice q . Después continúe su caminata W , momentáneamente interrumpida, hasta que alcance por primera vez un vértice de C_2 , lo cual seguramente ocurrirá porque G es conexo, etc., y la demostración queda completa. $\square$

Es extremadamente difícil, desde el punto de vista computacional, decidir si un grafo dado tiene un camino o circuito de Hamilton. Veremos en el Capítulo 5 que esta cuestión es típica de una clase de problemas que son el tema principal de ese capítulo, y son quizá los problemas sin resolver más (in)famosos en la informática teórica. Gracias al Teorema de Euler (Teorema 1.5), es fácil decidir si un grafo tiene un camino o circuito euleriano.

A continuación, nos gustaría tratar el coloreo de grafos, sin duda una de las partes más bonitas de la teoría de grafos. Suponga que hay K colores disponibles para nosotros, y que se nos presenta un grafo G . Un coloreo propio de los vértices de G es una asignación de un color a cada vértice de G de tal manera que $\forall e \in E(G)$ los colores de los dos extremos de e sean diferentes. La Figura 1.6(a) muestra un grafo G y un intento de colorear adecuadamente sus vértices en 3 colores ('R', 'Y' y 'B'). El intento falló porque una de las aristas de G tiene el mismo color asignado a ambos extremos. En la Figura 1.6(b) mostramos el mismo grafo con un coloreo propio exitoso de sus vértices en cuatro colores.

El número cromático $\chi(G)$ de un grafo G es el número mínimo de colores que pueden usarse en un coloreo propio de los vértices de G . Un grafo bipartito es un grafo cuyo número cromático es ≤ 2 , es decir, es un grafo que puede colorearse con dos colores. Eso significa que los vértices de un grafo bipartito pueden dividirse en dos clases 'R' y 'Y' tales que ninguna arista del grafo corre entre dos vértices 'R' ni entre dos vértices 'Y'. Los grafos bipartitos suelen dibujarse, como en la Figura 1.7, en dos capas, con todas las aristas yendo entre capas.

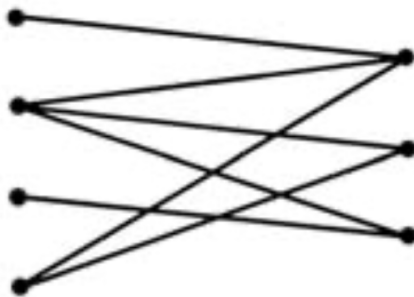


Figura 9: Un grafo bipartito.

El complemento $\overline{G}$ de un grafo G es el grafo que tiene el mismo conjunto de vértices que G y tiene una arista exactamente donde G no tiene sus aristas. Formalmente:

$$E(\overline{G}) = \{(v, w) \mid v, w \in V(G); v \neq w; (v, w) \notin E(G)\}.$$

A continuación se presentan algunas familias especiales de grafos que aparecen con tanta frecuencia que merecen nombres especiales. El grafo completo K_n es el grafo de n vértices en el que cada una de las $\binom{n}{2}$ aristas posibles está presente. Así, K_2 es una única arista, K_3 parece un triángulo, etc. El grafo vacío $\overline{K}_n$ consiste en n vértices aislados; es decir, no tiene aristas en absoluto.

El grafo bipartito completo $K_{m,n}$ tiene un conjunto S de m vértices y un conjunto T de n vértices. Su conjunto de aristas es $E(K_{m,n}) = S \times T$. Tiene $|E(K_{m,n})| = mn$ aristas. El n -ciclo, C_n , es un grafo de n vértices conectados para formar un único ciclo. Un árbol es un grafo que (a) es conexo y (b) no tiene ciclos. Se muestra un árbol en la Figura 1.8.

No es difícil probar que las siguientes son descripciones equivalentes de un árbol.

- (a) Un árbol es un grafo que es conexo y no tiene ciclos.
- (b) Un árbol es un grafo G que es conexo y para el cual $|E(G)| = |V(G)| - 1$.
- (c) Un árbol es un grafo G con la propiedad de que entre cada par de vértices distintos hay un camino *único*.

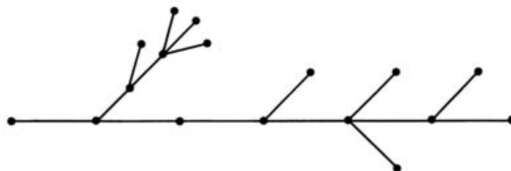


Figura 10: Un árbol.



Figura 11: Tres grafos etiquetados ...

Si G es un grafo y $S \subseteq V(G)$, entonces S es un conjunto independiente de vértices de G si ningún par de vértices en S es adyacente en G . Un conjunto independiente S es maximal si no es un subconjunto propio de otro conjunto independiente de vértices de G . Dualmente, si un subconjunto de vértices S induce un grafo completo, entonces hablamos de un subgrafo completo de G . Un subgrafo completo maximal de G se llama una *clique*.

Un grafo puede estar etiquetado o no etiquetado. Los vértices de un grafo etiquetado están numerados $1, 2, \dots, n$. Una diferencia que esto implica es que hay muchos más grafos etiquetados que grafos no etiquetados. Existen, por ejemplo, 3 grafos etiquetados que tienen 3 vértices y 1 arista, pero solo un grafo no etiquetado de ese tipo (Figuras 1.9 y 1.10).

La mayoría de los problemas de conteo en grafos son mucho más fáciles para grafos etiquetados que para grafos no etiquetados. Considere la siguiente pregunta: ¿Cuántos grafos hay que tienen exactamente n vértices?

Supongamos primero que hablamos de grafos etiquetados. Un grafo con n vértices tiene como máximo $\binom{n}{2}$ aristas. Para construir un grafo, decidiríamos cuáles de estas aristas posibles se usarán. Podemos tomar cada una de estas $\binom{n}{2}$ decisiones de manera independiente, y para cada forma de decidir dónde poner las aristas, obtendríamos un grafo distinto. Por lo tanto, el número de grafos etiquetados con n vértices es $2^{\binom{n}{2}} = 2^{n(n-1)/2}$.

Si formuláramos la pregunta correspondiente para grafos no etiquetados, veríamos que es muy difícil. La respuesta es conocida, pero la derivación involucra el lema de Burnside sobre la acción de un grupo en un conjunto y algunos argumentos de conteo bastante delicados.

Daremos la respuesta aproximada a esta pregunta, que es fácil de escribir, en lugar de la respuesta exacta, que no lo es. Si g_n es el número de grafos no etiquetados con n vértices, entonces: $g_n \sim \frac{2^{\binom{n}{2}}}{n!}$.



Figura 12: ... pero solo un grafo no etiquetado.

5.1. Ejercicios

1. Demuestre que un árbol es un grafo bipartito.
2. Halle el número cromático del n -ciclo.
3. Describa cómo determinaría, en una computadora, si un grafo dado G es bipartito.
4. Dado un entero positivo K , encuentre dos grafos diferentes, cada uno de cuyos números cromáticos sea K .
5. ¿Cuántos grafos etiquetados de n vértices y E aristas hay exactamente?
6. ¿En cuántos grafos etiquetados de n vértices los vértices $\{1, 2, 3\}$ forman un conjunto independiente?
7. ¿Cuántas cliques tiene un n -ciclo?
8. Verdadero o falso: un *ciclo hamiltoniano* es un ciclo inducido en un grafo.
9. ¿Qué grafo de n vértices tiene el mayor número de conjuntos independientes? ¿Cuántos tiene?
10. Dibuje todos los grafos conexos no etiquetados de 4 vértices.
11. Sea G un grafo bipartito que tiene q componentes conexas. Demuestre que hay exactamente 2^q maneras de colorear apropiadamente los vértices de G con 2 colores.
12. Encuentre un grafo G de n vértices, distinto del grafo completo, cuyo número cromático sea igual a 1 más el grado máximo de cualquier vértice de G .
13. Sea n múltiplo de 3. Considere un grafo etiquetado G que consta de $n/3$ componentes conexas, cada una de ellas un K_3 . ¿Cuántos conjuntos independientes *maximales* tiene G ?
14. Describa el complemento del grafo G del Ejercicio 13 anterior. ¿Cuántas cliques tiene?
15. ¿En cuántos grafos etiquetados de n vértices el subgrafo inducido por los vértices $\{1, 2, 3\}$ es un triángulo?
16. Sea H un grafo etiquetado de L vértices. ¿En cuántos grafos etiquetados de n vértices el subgrafo inducido por los vértices $\{1, 2, \dots, L\}$ es igual a H ?

17. Diseñe un algoritmo que decida si un grafo dado, de n vértices y m aristas, contiene o no un triángulo, en tiempo $O(\max(n^2, mn))$.
18. Pruebe que el número de grafos etiquetados de n vértices, todos cuyos vértices tienen grado par, es igual al número de todos los grafos etiquetados de $n - 1$ vértices.